TUTORIAL · EDITION 2 · 19 OCTOBER 2026 · ACI + EVPN PLATFORM

How the platform works, end to end

From a sentence typed to an AI agent to configuration running on a Cisco APIC: every component, every check, what each check proves — and, new in this edition, the LangGraph reasoning layer and a walk-through of one lab.

WHAT IT IS

The platform turns **intent** ("give the sales team a tenant with a VRF and two bridge domains") into **network configuration**, and then proves the configuration is really there. Nobody types CLI commands or clicks through the APIC GUI.

It is built from ordinary parts. An **AI agent** talks to an **MCP Server**. The server records intent in **Nautobot**, the source of truth. A change there fires a **webhook** that starts the **GitLab CI pipeline**, a chain of jobs that runs on the **GitLab Runner** and carries the intent from Nautobot to the APIC. The pipeline renders the intent into YAML, checks it, plans it with **Terraform**, waits for a person to approve, takes an **APIC backup**, applies the change, runs **Ansible** and **pyATS**, checks APIC faults, and writes the evidence back.

New since September: a **reasoning layer built with LangGraph** sits beside the agent. It only reads. It explains why something did not reach the APIC, and what a pending change will do before anyone approves it.

The whole idea in four lines

Nautobot holds *what should be true*.
Git holds *every version of it*.
The pipeline makes the network *match it*.
Tests and checks produce *evidence*, and the evidence says how strong it is.

The AI agent and the reasoning layer are optional: the pipeline works the same whatever made the change.

THE LINE-UP — 17 COMPONENTS YOU WILL MEET

- **AI Agent** — MCP client: VS Code Copilot, Claude
- **Reasoning layer** — LangGraph, read-only: diagnose, review-plan
- **MCP Server** — 138 tools, the agents' only hands
- **Nautobot** — source of truth for intent
- **Webhook** — Nautobot tells GitLab "something changed"
- **GitLab (VCS)** — Git repository, history, audit trail
- **GitLab CI + Runner** — the pipeline and the worker running it
- **Generator** — renders Nautobot data into NetAsCode YAML
- **OPA** — policy engine: is this intent allowed?
- **Vault** — hands out APIC credentials at run time
- **Terraform** — infrastructure as code: plan, then apply
- **Terraform state** — kept in GitLab, so plans show real change
- **Ansible** — Day-2 tasks on existing objects
- **pyATS** — Cisco's test framework: reads the network back
- **Fault check** — did APIC flag anything new as invalid?
- **MinIO** — S3-compatible artifact & knowledge store
- **Cisco APIC** — the fabric controller: observed state

WHAT THIS TUTORIAL COVERS

★ What's new
1 The map
2 The protocols
3 One change, start to finish
4 Each component
5 The concepts
6 Integrity & checks
7 Testing & evidence
8 API design
9 LangGraph
10 Running a lab
G Glossary

★ What changed since the September edition

Edition 1 described the platform on 25 September 2026. Since then the tool catalogue has more than doubled, the pipeline has gained a backup step and a memory, and a reasoning layer has been added. Changed pages are updated in place; sections 9 and 10 are new.

Date (2026)	Added or fixed	Why it matters	Evidence
12 Oct	**Read / update / delete tools** (11), **port-channel, vPC, node-management** (12), and the **NOT-SETTABLE catalogue**	Intent can be inspected, changed and removed through MCP, and every setting the platform cannot set yet is named (26 open).	LIVE
13–16 Oct	**Fabric-wide policies** (19 tools), **BGP route reflector** (4), **ESG and vzAny** (10), **L3Out depth** (14)	Most of the ACI configuration the labs need is now expressible as intent.	LIVE
17 Oct	**APIC configuration backup before every apply**	If an apply goes wrong, the fabric can be restored to the moment before. No backup, no apply.	LIVE restore tested
17–18 Oct	**LangGraph reasoning layer**: `diagnose`, then review fixes	Explains, layer by layer, why an object is not (validly) on the APIC.	LIVE
18 Oct	**Terraform state kept in GitLab**	Before, every plan said "create everything". Now a plan shows only real changes, and shows drift.	LIVE
18 Oct	**Approval assistant**: `review-plan` and the MCP tool `get_plan_changes`	Before anyone approves, it says in plain words what the plan creates, changes and deletes, and warns if the plan is outdated.	LIVE
18 Oct	**Fix: plans now read this pipeline's YAML**	Found by the approval assistant: every plan had been one pipeline behind Nautobot.	LIVE ACI · UNIT EVPN
18–19 Oct	**Fix: VRF descriptions now reach the APIC**	Also found by the approval assistant: the module had never sent them.	LIVE applied

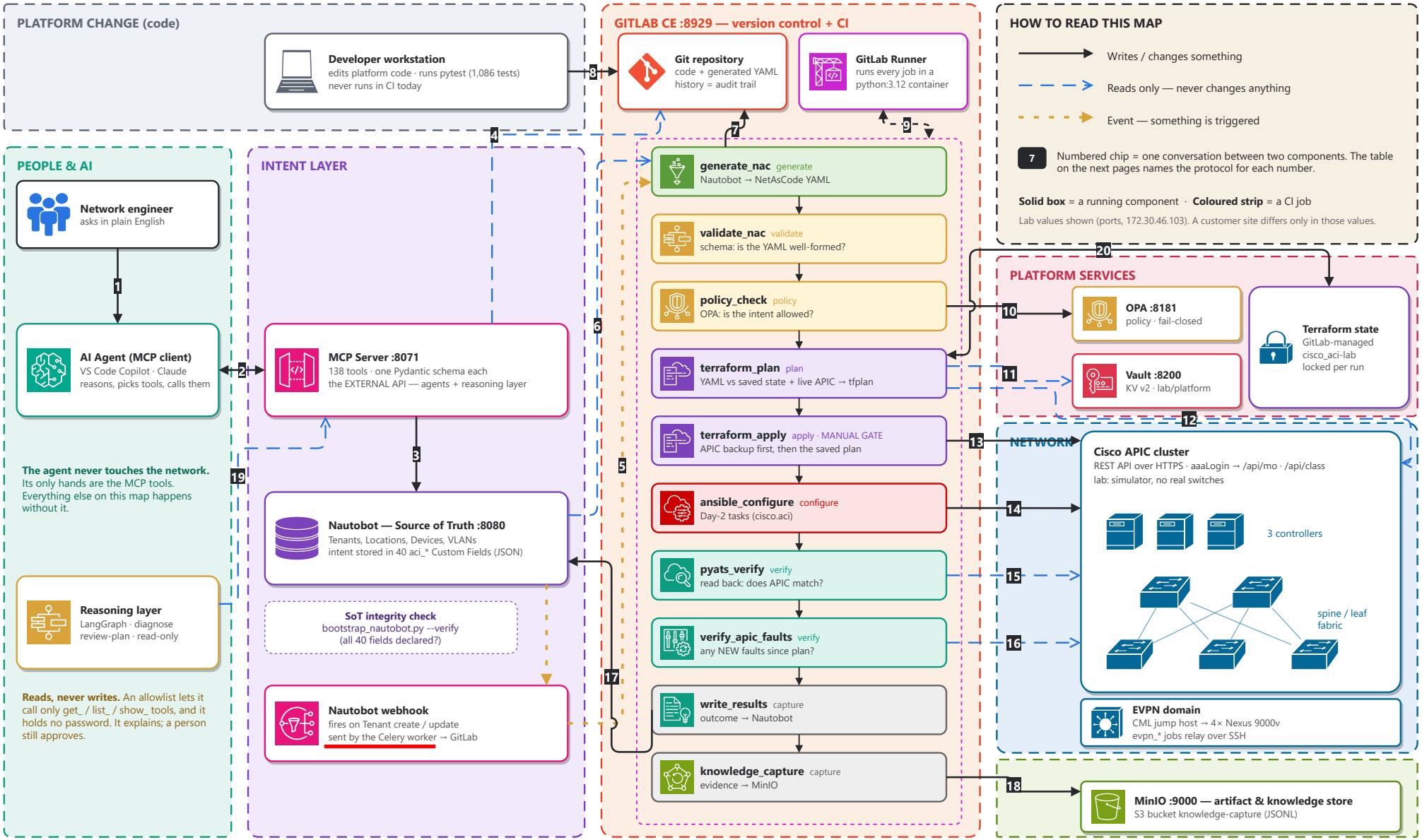
The numbers, then and now

Measure	25 September	19 October
MCP tools (live-verified)	65 (35)	**138 (109)**: 133 ACI, 3 EVPN, 2 generic
Nautobot Custom Fields (the intent schema)	32	**40**
Terraform resource blocks in the ACI module	88	**113** (98 resource types)
Automated tests	544	**1,086**: 244 platform + 785 MCP Server + 57 reasoning
ACI Terraform state	none kept between pipelines	**GitLab-managed**, state `cisco_aci-lab`
Backup before apply	none	**every apply**, fail-closed
Reasoning workflows	none	**2**, both read-only

PLEASE!/ NOTE

Still open

No scheduled drift detection; no MCP authentication; in this lab the webhook is not delivered (Celery worker down) and targets `main`, which lags the work branch. Section 10 shows the workaround.



Read it left to right. A person talks to the agent (1). The agent calls MCP tools (2), which write intent into Nautobot (3). Nautobot's webhook starts the pipeline (5). Every CI job then runs, top to bottom, on the GitLab Runner (9), each talking to one service or to the APIC. The **top band** is the other way the platform changes: a developer changing the platform's own code (8). **New:** the reasoning layer reads through the MCP Server (19), and the plan and apply jobs keep Terraform's state in GitLab (20). The numbers match the protocol table on the next two pages.

2 What travels on each arrow

Everything in this platform talks **HTTP(S)**, with two exceptions: Git itself and the SSH relay to the Nexus lab. The differences are in *what is inside* the HTTP request: JSON-RPC, REST, GraphQL, a webhook body, or the S3 API. Type: **write** changes something, **read** only looks, **event** triggers something.

#	From → To	Protocol	What travels, and why it matters
1	Engineer → AI Agent	Natural language (chat)	**write** A request in plain English. The only step with no fixed format.
2	AI Agent ⇄ MCP Server	**MCP**: JSON-RPC 2.0 over **streamable HTTP**, port 8071	**write** `tools/list` tells the agent which tools exist and their JSON Schemas. `tools/call` invokes one with typed arguments. The reply is text the agent reads.
3	MCP Server → Nautobot	**REST** over HTTP(S), via `pynautobot`, token auth	**write** Creates Tenants/VRFs and PATCHes intent into `aci_*` Custom Fields as JSON. **Watch out:** an undeclared field returns HTTP 200 and keeps nothing.
4	MCP Server → GitLab	**GitLab REST API** (`python-gitlab`)	**read** `show_status`, `get_pipeline_report`, `get_generated_intent` and `get_plan_changes` read pipelines, artifacts and job logs. The MCP Server never starts or approves a pipeline.
5	Nautobot → GitLab CI	**Webhook**: HTTP POST to `/projects/:id/trigger/pipeline`	**event** Fires when a `tenancy.tenant` is created or updated. Sent by Nautobot's **Celery worker**, with a trigger token, for branch `main`. No listener service sits in between.
6	generate_nac → Nautobot	**GraphQL** over HTTP, `POST /api/graphql/`	**read** One query fetches exactly the fields the generator needs. REST is used for writes and GraphQL for reads.
7	generate_nac → Git repository	**Git** over HTTP(S): `git push`	**write** Commits the rendered `tenants.yaml` with `[skip ci]`. Later jobs must take the file as an **artifact** of this job, not from Git: Git still holds the previous pipeline's copy (the bug fixed on 18 October).
8	Developer → Git repository	**Git** over HTTPS/SSH	**write** Platform code changes: tools, generator, Terraform, playbooks. Fast-forward merges only.
9	GitLab Runner ⇄ GitLab	HTTPS: the Runner **polls** for jobs and uploads artifacts	**write** GitLab never pushes work to the Runner; the Runner asks for it. Each job runs in a fresh `python:3.12-slim` container.

#	From → To	Protocol	What travels, and why it matters
10	policy_check → OPA	**REST**: `POST /v1/data/platform/<domain>/decision`, port 8181	**read** Sends the intent and gets allow/deny with reasons. **Fail-closed**: if OPA cannot be reached, the job fails (ADR-014).
11	terraform_plan/apply → Vault	Vault **HTTP API**, `X-Vault-Token`, KV v2, port 8200	**read** `load-vault-creds.sh` reads `secret/data/lab/platform` and exports `TF_VAR_aci_*`, plus the backup passphrase. No password is ever in Git.
12	terraform_plan → APIC	**APIC REST** over HTTPS (GET)	**read** The plan **refreshes** every object in the saved state from the live APIC, then compares the result with the YAML. That is how it notices an object changed or deleted behind its back. Also takes the fault "before" snapshot.
13	terraform_apply → APIC	**APIC REST** over HTTPS: `aaaLogin`, then `POST /api/mo/…`	**write** First `apic_backup.py` takes an APIC configuration snapshot and waits for it; no snapshot, no apply. Then the CiscoDevNet/aci provider turns each resource into managed-object (MO) calls addressed by DN. Runs only after a human clicks Run.
14	ansible_configure → APIC	**APIC REST** over HTTPS (`cisco.aci` modules)	**write** Day-2 tasks (`aci_tenant`, `aci_ap`, `aci_epg`). Ansible does not create new infrastructure; Terraform does.
15	pyats_verify → APIC	**APIC REST** (pyATS `rest.connector`, `os: apic`)	**read** Reads tenants and VRFs back and compares them with `tenants.yaml`.
16	verify_apic_faults → APIC	**APIC REST**: class query on `faultInst`	**read** Compares faults against the snapshot taken at plan time and fails on any *new* fault on a managed object.
17	write_results → Nautobot	**REST**: `PATCH /api/tenancy/tenants/<id>/`	**write** Puts the pipeline outcome back on the tenant, so Nautobot shows whether its intent actually deployed.
18	knowledge_capture → MinIO	**S3 API** (`boto3`), port 9000	**write** Appends one JSONL record per run: intent, commit, pipeline, outcome.
19	Reasoning layer → MCP Server	**MCP**, as in row 2, through a **read-only allowlist**	**read** Only 7 tools can be called: `show_status`, `list_aci_tenants`, `get_tenant_intent`, `get_fabric_intent`, `get_generated_intent`, `get_pipeline_report`, `get_plan_changes`. Any other name is refused *before* a request leaves the process. The layer holds no password.

20	terraform_plan/apply ⇄ GitLab Terraform state	Terraform **http backend**; `CI_JOB_TOKEN` ; lock with `POST` , unlock with `DELETE`	**write** Each plan and apply locks the state `cisco_aci-lab` , reads it, and the apply saves it back. No new secret: the job's own token authenticates. **The state contains credentials** (SNMP community strings, local-user passwords), so project membership grants access to them.

EVPN domain: the GitLab Runner has no network path to the Nexus devices, so the `evpn_*` jobs relay through a CML jump host over SSH (driven by `expect` scripts). EVPN Terraform state persists on the jump host.

> **"The pipeline", precisely**
>
> Wherever this tutorial says *the pipeline*, it means the **GitLab CI pipeline** defined in `pipelines/aci.gitlab-ci.yml` : a chain of jobs that **GitLab** schedules and the **GitLab Runner** executes. It starts from the intent in **Nautobot** (read by `generate_nac`) and ends at the **Cisco APIC** (written by `terraform_apply` , read back by the verify jobs). It does not include the agent, the MCP Server or the reasoning layer: those only *cause* it (through Nautobot) or *read* it.

GraphQL over http

3 The journey of one change

An engineer types: "*Create a tenant called sales with a VRF sales-vrf and a bridge domain web-bd on 10.0.1.254/24.*" Here is everything that happens next.

1 **The agent picks tools.**
It has already called `tools/list`, so it knows `create_tenant`, `create_vrf` and `create_bridge_domain` exist, and each one's argument schema. It decides the order: tenant first, because the others belong to it.

2 **The MCP Server validates each call before doing anything.**
Pydantic checks the arguments against the tool's schema: types, required fields, naming rules, cross-field rules. Invalid input is refused with a readable error, and nothing is written.

3 **Intent lands in Nautobot.**
The server creates the Tenant (`ACI:sales`) and writes the VRF and bridge domain over REST. From here on, the record in Nautobot is the definition of what the network should contain.

4 **The webhook fires and the pipeline starts.**
Nautobot sees a tenant change and POSTs to GitLab's pipeline-trigger API (or a person presses **Run pipeline**). GitLab creates the pipeline; the Runner starts picking up its jobs.

5 **generate_nac renders the intent.**
The generator queries Nautobot over GraphQL, refuses dangling references, writes `tenants.yaml`, commits it with `[skip ci]` and hands the file on as an **artifact**. Every later job downloads this exact file.

6 **validate_nac and policy_check.**
Is the YAML well-formed? Does OPA allow it (naming, the protected system tenants `common`, `infra`, `mgmt`)? If OPA is down the answer is **no**. Open Policy Agent (OPA) policy validation uses the declarative Rego language to evaluate structured data (such as JSON, Kubernetes manifests, or API requests) and return pass/fail or allow/deny decisions.

7 **terraform_plan works out the difference.**
Credentials come from Vault. Terraform locks and reads its saved **state** from GitLab, refreshes each object from the APIC, compares with the YAML, and saves the exact list of changes as `tfplan`. It also snapshots APIC faults.

PLEASE!! NOTE

8 **Someone reviews the plan.**
Optionally, and recommended: `review-plan` (section 9) summarises it in plain words and warns if a newer pipeline has planned since.

9 **A person approves; a backup is taken first.**
`terraform_apply` is a **manual job**. When someone clicks Run, it first takes an APIC configuration snapshot, then applies *that saved plan* and nothing else, and saves the new state.

10 **ansible_configure runs Day-2 tasks.**
Idempotent operations on objects that now exist.

11 **Two independent verifications.**
`pyats_verify` reads objects back from the APIC; `verify_apic_faults` asks whether APIC raised any *new* fault on a managed object, because APIC can accept configuration and then mark it invalid.

12 **Evidence is written down.**
`write_results` writes the outcome onto the Nautobot tenant; `knowledge_capture` stores a JSONL record in MinIO. The agent's `show_status`, or `diagnose`, can now report what happened.

> **Why so many steps for one tenant?**
> Each step catches a different class of mistake, and each is cheap next to a misconfigured production fabric. Every step also leaves an artifact behind (the YAML commit, the plan, the backup record, the job logs, the fault delta, the JSONL record), so afterwards you can prove what happened and who approved it.

4 The components, one by one

THE REQUESTER

AI Agent (MCP client)

What A language model inside VS Code Copilot or Claude that can call tools. It reasons about a request and decides which tools to call, in which order, with which arguments.

Here Configured in `.vscode/mcp.json`. The business-facing persona `network-platform-operator` has no terminal *by design*: its only way to change anything is an MCP tool.

Talks MCP (JSON-RPC 2.0 over streamable HTTP).

Watch The agent can misunderstand. Every guard sits *behind* it, so a wrong call is refused or caught, never trusted.

THE EXTERNAL API

MCP Server

What MCP (Model Context Protocol) is an open standard for giving AI models tools. The server publishes each tool with a name, a description and a JSON Schema for its arguments.

Here `mcp-server/`, port 8071. **138 tools** (133 ACI, 3 EVPN, 2 generic) to create, read, update and delete intent, each backed by one Pydantic schema. Every tool declares its **evidence level** and the settings it **cannot set yet** (NS ids) in code.

Talks MCP in; Nautobot REST out (writes); GitLab REST out (read-only).

Watch `MCP_API_KEY` is declared in config but nothing checks it: the endpoint is unauthenticated today. It must be fixed before a customer deployment.

THE SOURCE OF TRUTH

Nautobot

What An open-source network source-of-truth (SoT) application: a database of what the network *should* be, with a REST API, a GraphQL API and webhooks.

Here Port 8080. Standard objects (Tenant, Location, Device, VLAN, Prefix, VRF) hold the skeleton; **40** `aci_*` **Custom Fields** hold the ACI-specific intent. A **Location** holds fabric-wide intent and never reaches the APIC as an object itself.

Talks REST (writes from MCP and write_results), GraphQL (reads by the generator), webhook out.

the generator performs read action upon nautobot webhook out

Watch The schema must be declared first: `bootstrap_nautobot.py`. Otherwise writes vanish with HTTP 200.

THE TRIGGER

Nautobot Webhook

What A built-in Nautobot feature: when an object of a chosen type changes, send an HTTP request to a URL. Nautobot's background **Celery worker** sends it.

Here Registered on `tenancy.tenant` (create and update); posts to GitLab's pipeline-trigger API for branch `main`.

Talks HTTP POST, one way. It does not wait for or read the result.

Watch Changes on a VRF, Prefix or Location do not fire it. In this lab the Celery worker has been down since mid-September, so no webhook is delivered at all until it is restarted.



VERSION CONTROL

GitLab CE — the Git repository

What A VCS (version control system) records every version of every file, who changed it and why. GitLab hosts the Git repository and adds a web UI, API and CI.

Here Port 8929, project `root/nautobot-infra-automation`. Holds the platform code *and* every rendered `tenants.yaml`, so the history of the network's intent is a Git log.

Talks Git over HTTP(S)/SSH; REST API.

Watch Fast-forward merges only; the history is the audit trail, so never rewrite it.

THE ENGINE

GitLab CI pipeline + Runner

What CI (continuous integration) runs a defined sequence of **jobs** in **stages**. The **Runner** is the worker process that executes them.

Here generate → validate → policy → plan → apply → configure → verify → capture. Shared templates in `pipelines/includes/common.gitlab-ci.yml`; domain wiring in `aci.gitlab-ci.yml` and `evpn.gitlab-ci.yml`.

Talks Runner polls GitLab over HTTPS; jobs pass files as **artifacts**.

Watch `needs:` is not transitive. Until 18 October the plan job did not list `generate_nac`, so it planned last pipeline's YAML. A test now enforces this.



CONFIG RENDERING

Generator (Nautobot → NetAsCode YAML)

What Python that reads intent from Nautobot and writes it in the **NetAsCode** YAML format, the authoritative input for Terraform (ADR-018).

Here `platform/python/generator/` : `client.py` (GraphQL) and `transformer.py` (the rules). Output is deterministic and committed.

Talks GraphQL in; a file out; Git push.

Watch It is also an **integrity gate**: dangling references raise `PolicyReferenceError`, unknown fabric nodes raise `FabricInventoryError`.

VALIDATION & POLICY

validate_nac + OPA

What `validate_nac.py` checks the YAML's form. **OPA** (Open Policy Agent) evaluates rules written in its Rego language against the intent and returns a decision.

Here `policy_check.py` POSTs to OPA on port 8181. A request to recreate a system tenant is denied.

Talks REST.

Watch Validation answers "is it well-formed?"; policy answers "is it allowed?". Neither answers "will it work?"

SECRETS

HashiCorp Vault

What A secrets manager: stores credentials encrypted and releases them to callers holding a valid token.

Here Port 8200, KV v2 path `secret/lab/platform` : APIC credentials and the APIC backup passphrase (`apic_export_passphrase`), exported just before Terraform runs.

Talks Vault HTTP API with `X-Vault-Token` .

Watch Lab Vault is re-initialised on each start; the root token is regenerated.

INFRASTRUCTURE AS CODE

Terraform

What Declares *what should exist*. `plan` computes the difference from reality; `apply` makes the changes. Keeps a **state** mapping each declared resource to a real object.

Here `platform/terraform/aci/main.tf` : **113 resource blocks** (98 types) iterating over the YAML with `for_each` . Provider `CiscoDevNet/aci` . State in GitLab.

Talks APIC REST over HTTPS; GitLab's state API.

Watch A clean `plan` checks the provider's schema, not APIC's rules. Relations need a **DN**, not a name. An attribute the module never sets is silently ignored (VRF descriptions, until 18 October).

DAY-2 OPERATIONS

Ansible

What Runs idempotent tasks from YAML **playbooks** through modules; each module knows one kind of object.

Here `platform/ansible/aci/playbooks/day2-epg.yml`, `verify-tenants.yml`; `cisco.aci` and `community.hashi_vault` collections.

Talks APIC REST over HTTPS.

Watch Ansible never provisions new infrastructure here. That is Terraform's job, so the two never fight over the same object.

POST-DEPLOYMENT VERIFICATION

pyATS

What Cisco's Python test framework for networks: a **testbed** file describes devices and connections; test scripts connect, read, and assert.

Here `tests/pyats/aci/`: `job.py` runs `test_aci_tenants.py` and `test_aci_vrfs.py` in CI. EVPN uses a plain-Python equivalent (no `genie` on the jump host).

Talks APIC REST, read-only.

Watch pyATS is **not** pytest. It tests the *network*; pytest tests the *code*.

VALIDITY CHECK

APIC fault delta

What APIC raises a **fault** on any object whose configuration it considers invalid, even after accepting it.

Here `check_apic_faults.py --snapshot` at plan time, `--compare` after apply, scoped to managed tenants plus fabric-wide objects this platform owns.

Talks APIC REST class query on `faultInst`.

Watch Take the snapshot *before* the change, or every rebuilt object looks like a new fault.

ARTIFACT & KNOWLEDGE STORE

MinIO (+ write_results)

What An S3-compatible object store: files in buckets, reached through the S3 API.

Here Port 9000, bucket `knowledge-capture`: one JSONL line per run. `write_results` also stamps the outcome on the Nautobot tenant.

Talks S3 API (`boto3`); Nautobot REST.

Watch GitLab artifacts expire (1 week to 1 month). MinIO is where records are kept for longer.

THE OBSERVED STATE

Cisco APIC (and the Nexus EVPN lab)

What The APIC is the controller of an ACI fabric. Everything in it is a **managed object (MO)** with a unique **distinguished name (DN)**, e.g. `uni/tn-sales/ctx-Presales_VRF`, readable and writable over one REST API.

Here A simulator at `172.30.46.103` with **no real switches**, so physical paths stay `unformed` and no traffic ever flows. It still accepts almost all configuration, so far more can be live-verified than was once assumed. EVPN runs on 4 real Nexus 9000v in CML.

Watch "Apply succeeded" is not "the config is valid". Always check faults.

EXPLAINS, NEVER CHANGES

Reasoning layer (LangGraph)

What A small Python program, `reasoning/` (package `platform_reasoning`), whose workflows are LangGraph graphs. Section 9 explains it from first principles.

Here Two read-only workflows: `diagnose` (why is this object not, or not validly, on the APIC?) and `review-plan` (what will approving this pipeline change?). Own virtual environment `reasoning/.venv`; 57 tests.

Talks MCP to the MCP Server, limited to 7 read tools; reads `knowledge/` from disk.

Watch It holds no password and cannot start or approve a pipeline (ADR-022 Decision 6). No AI model makes any decision in it today.

TERRAFORM'S MEMORY

Terraform state store (GitLab)

What Terraform's **state** is its record of which real object belongs to which declared resource. Without it Terraform cannot tell a change from a rebuild.

Here GitLab's built-in Terraform state store, project 1, state `cisco_aci-lab` (*Operate › Terraform states*). Written only by CI. Adopted on 18 October: pipeline 63 recorded the 116 existing objects; pipeline 65 then planned "No changes".

Talks Terraform http backend; job token; per-run lock.

Watch It holds credentials (SNMP community strings, local-user passwords). Local test runs use their own state and must never point at this one.

THE UNDO BUTTON

APIC configuration backup

What An APIC configuration snapshot (an encrypted export of the whole fabric configuration) taken immediately before Terraform changes anything.

Here `apic_backup.py --create` in `terraform_apply`; the record is the artifact `apic-backup.json`; the newest 20 snapshots are kept. Restore runbook: `ACI-Restore-From-Pre-Apply-Backup.md`.

Talks APIC REST (`configExportP`); passphrase from Vault.

Watch Fail-closed: no completed snapshot, no apply. APIC 6.2 refuses every export until a global AES passphrase is set.

HONEST GAPS, IN CODE

NOT-SETTABLE catalogue

What A list of the settings the APIC supports but the platform cannot set yet, each with an id (`NS-xx`), why, what applies meanwhile, and the next step.

Here `mcp-server/src/mcp_server/not_settable.py`, 26 open entries. Each id is printed in the description of every tool it affects, marked on the Terraform resource, and listed in `Evidence-Matrix.md`; a test fails if the four disagree.

Talks Read by the agent (tool descriptions) and by `review-plan` (caveats).

Watch A missing setting is otherwise invisible: the call succeeds and the APIC default silently applies.

READS FOR MACHINES

Three read-only MCP tools added for the reasoning layer

What Tools that return structured JSON about the pipeline, rather than prose, so a program can compute on the answer. Any MCP client can use them, including the Copilot agent.

Here `get_pipeline_report` (jobs, Terraform summary, parsed APIC fault delta), `get_generated_intent` (the exact YAML block a pipeline's Terraform consumed) and `get_plan_changes` (every resource a pending plan creates, changes, deletes or replaces, plus drift and the module's caveats). All three are live-verified against GitLab.

5 The concepts behind the parts

Infrastructure as Code (IaC)

Describing infrastructure in text files that a tool turns into real configuration, instead of configuring it by hand. The key property is that IaC is **declarative**: you say *what should exist*, and the tool works out how to get there. Applying the same files twice changes nothing the second time (**idempotence**). Here, Terraform is the IaC engine, and NetAsCode YAML is the declaration it reads.

Version control (VCS)

Git keeps every version of every file along with who changed it, when, and why. In this platform Git versions **two different things**: the platform's code, and every rendered `tenants.yaml`. The second one is unusual and valuable. `git log` shows the history of the network's intended state, `git diff` between two commits shows exactly what a pipeline changed, and **rollback** can mean re-applying an earlier YAML or restoring the pre-apply APIC backup.

Templates and scaffolding

A **template** is a reusable structure with blanks for data. A **scaffold** is the fixed skeleton that every instance shares. In this platform the Terraform module is the template: `main.tf` says "for every bridge domain in the YAML, create an `aci_bridge_domain` with these attributes", and never names a real bridge domain. Each MCP tool follows the same scaffold (schema class → client method → registry entry with an evidence level), which is why adding a tool is mechanical.

Config rendering

Rendering means combining a template with data to produce a concrete, reviewable output. It happens **twice** here, and both outputs are kept: the generator renders Nautobot data into `tenants.yaml`, and Terraform renders its module plus that YAML into `tfplan`. Keeping the rendered output is what makes changes reviewable before they happen, and **deterministic**: identical input must give byte-identical output, which `check_determinism.sh` enforces.

Artifacts and the artifact store

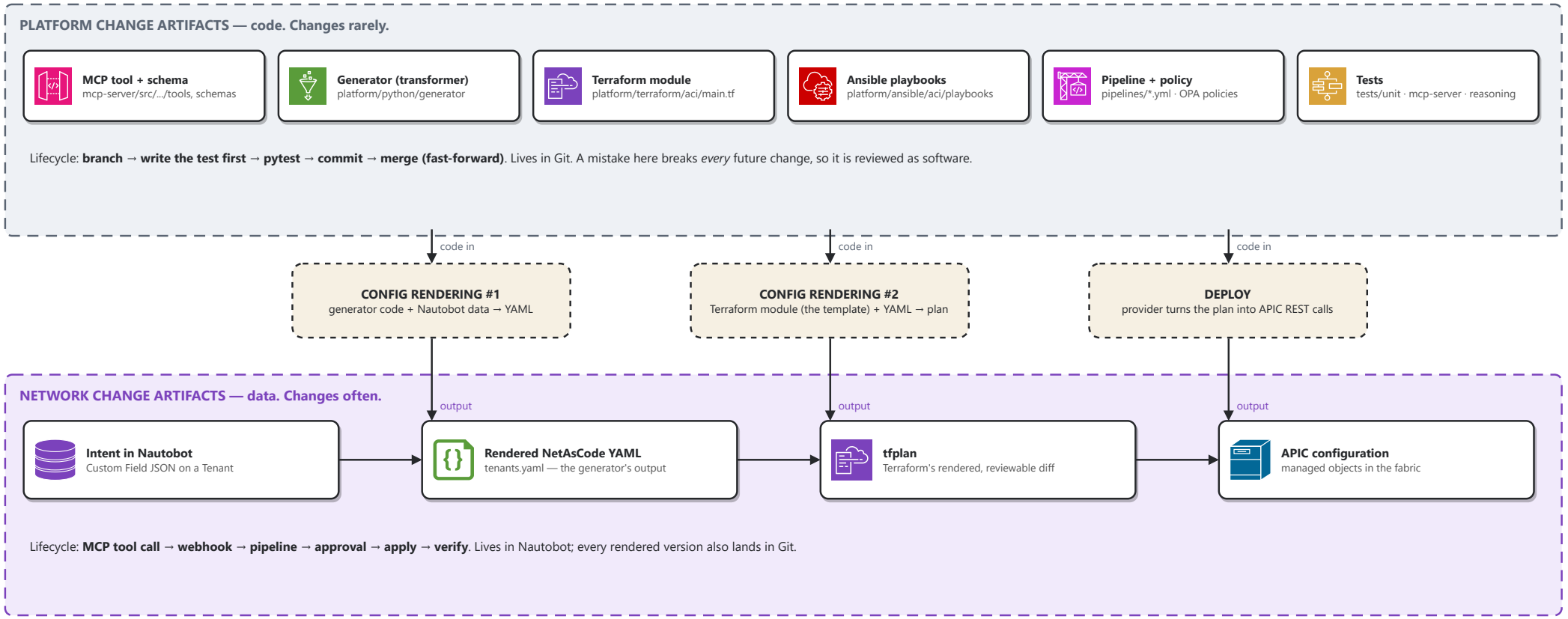
An **artifact** is a file a CI job produces and hands on: the YAML, the `tfplan`, the fault snapshot, the backup record, the knowledge record. GitLab passes artifacts only between jobs listed in `needs:`; a job that forgets to list one silently falls back to the Git copy. The plan is the most important artifact: `terraform_apply` applies the *saved plan*, not a fresh one, so what the approver saw is exactly what runs.

Pipeline runner

The **GitLab Runner** is the worker. It polls GitLab for pending jobs, starts a clean container for each one, runs its script, uploads artifacts and reports status. Because every job starts from a clean container, a job cannot rely on leftovers from an earlier one. Anything it needs must arrive as an artifact, from Git, from Vault or from the state store. A `resource_group` lock stops two pipelines applying to the same target at once.

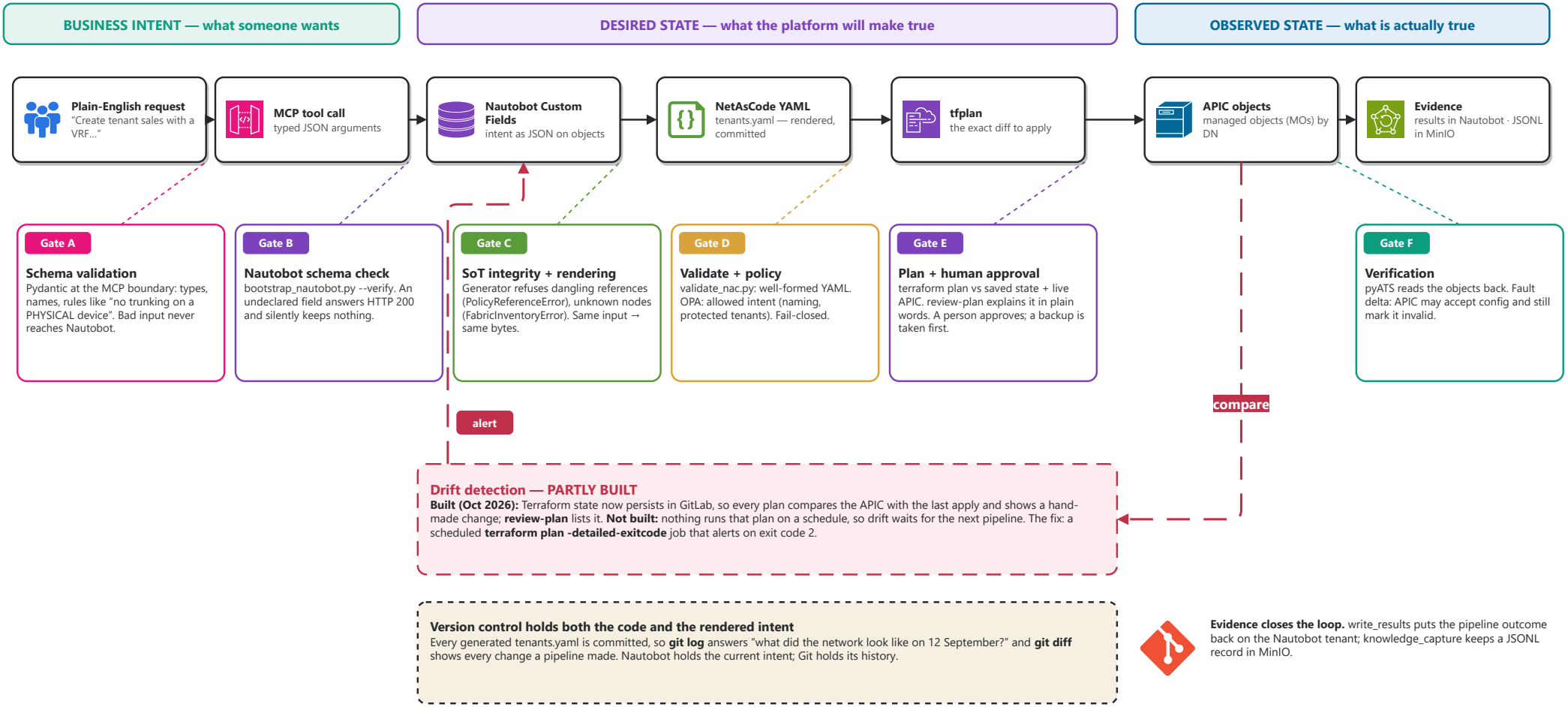
Integration and simulation

A **simulator** stands in for real equipment. The ACI simulator runs the real APIC software with no real switches: excellent for proving configuration is *accepted and valid*, unable to prove anything about *forwarding*. The EVPN side uses **emulation** (Nexus 9000v in Cisco Modeling Labs), which runs real NX-OS, so BGP/EVPN sessions really form. The limits of each environment are recorded, not assumed.



Platform change artifacts are the machine itself: MCP tools, the generator, the Terraform module, playbooks, pipeline definitions, policies, the reasoning layer and tests. They change rarely, and a mistake in one breaks *every* future network change. So they go through software discipline: a test is written first and watched fail, the suite runs, the change is committed and merged, and the ADR and status documents are updated (the Definition of Done in `CLAUDE.md`).

Network change artifacts are what the machine processes: intent in Nautobot, the rendered YAML, the plan, the resulting APIC objects. They change often and flow through the pipeline with its gates. The separation is deliberate. Adding a tenant never requires a code change, and changing code never silently changes a tenant, because every rendered result is diffable in Git.



The Three Truths (ADR-019). Business intent is what someone wants. **Desired state** is the platform's precise, machine-readable version of it. **Observed state** is what the network actually is. Every gate sits on a transition between two artifacts and checks one specific thing. The red box is the transition that is only partly gated: since the Terraform state persists, every plan notices drift, but nothing yet runs that check on a schedule.

6 Four ideas that are easy to confuse

SCHEMA VALIDATION

Question it answers: is this input the right *shape*?

It happens at three layers, each for a different schema. **At the MCP boundary**, Pydantic models check every tool call: types, required fields, value formats, and cross-field rules such as "trunking is invalid on a PHYSICAL L4-L7 device". **At Nautobot**, the *data* schema is the set of Custom Fields; `bootstrap_nautobot.py --verify` checks all 40 exist with the right type on the right object type. **At the YAML**, `validate_nac.py` checks the rendered file. Schema validation cannot tell you whether a well-formed request *makes sense*, for example whether the VRF it names exists.

SOURCE-OF-TRUTH (SOT) INTEGRITY CHECK

Question it answers: is the data in the source of truth *coherent*, and can it be trusted as the definition of the network?

- **Schema integrity:** every field the platform reads or writes is declared. A unit test fails if code references an `aci_*` field that `bootstrap_nautobot.py` does not declare.
- **Referential integrity:** the generator refuses intent that points at things that do not exist, such as a route map naming a missing match rule, or a bridge domain naming a missing L3Out (`PolicyReferenceError`).
- **Uniqueness:** tools deduplicate instead of appending. Two entries with the same name reach Terraform as a `Duplicate object key` error.
- **Determinism:** the same SoT content always renders the same YAML, so a diff in Git always means a real change in intent.

DRIFT DETECTION — PARTLY BUILT

Question it answers: has the network changed without the source of truth knowing?

Drift is any difference between desired and observed state that the platform did not cause: a change made by hand in the APIC GUI, an object deleted by another tool. **Built (18 October):** because Terraform's state now persists in GitLab, every plan compares the live APIC with the last apply and reports drift on the objects it manages, and `review-plan` lists it. **Not built:** nothing runs that plan on a schedule, so drift waits for the next pipeline, and objects Terraform does not manage are never compared. The fix is a scheduled pipeline running `terraform plan -detailed-exitcode`, where exit code 2 means "differences found" and raises an alert.

VERSION CONTROL (VCS) AS A CONTROL, NOT JUST STORAGE

Question it answers: what was the intent at any moment, and who changed it?

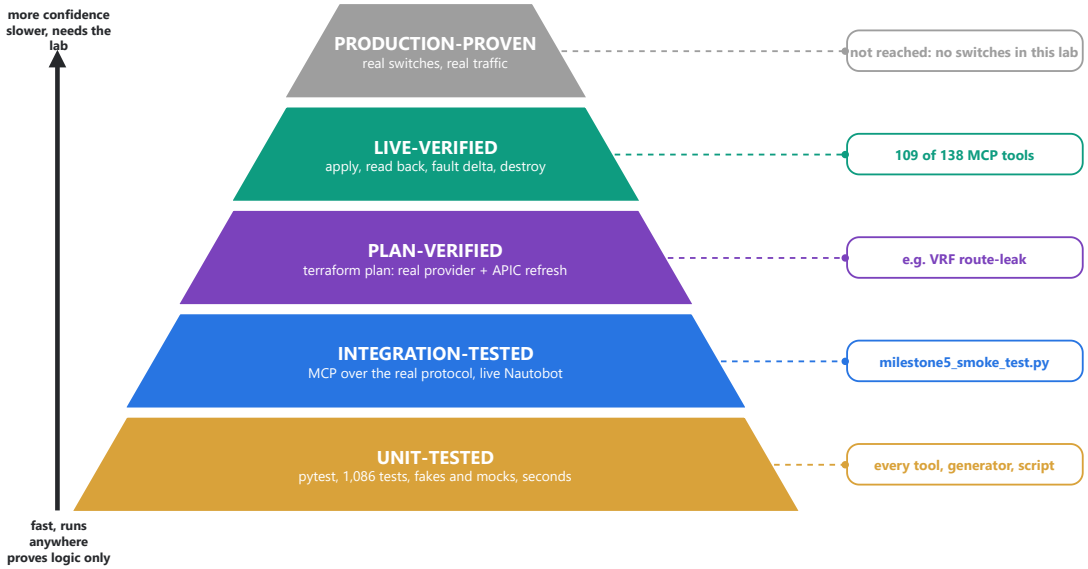
Nautobot holds only the *current* intent. Git holds its *history*, because every rendered YAML is committed. That gives an audit trail (commit, pipeline ID, approver), a review surface (the diff), and a rollback mechanism. The repository rules protect it: fast-forward merges only, never force-push, explicit `git add` paths, and no secrets or generated binaries committed.

7 How we know it works, and how sure we are

These four words mean different things. Mixing them up is the most common way a platform like this ends up claiming more than it has proven.

Word	Question it answers	In this platform
Testing	Does the *code* behave as designed?	pytest: 1,086 tests in three suites (platform 244, MCP Server 785, reasoning 57). They run on a developer machine before code is merged.
Validation	Is the *input or intent* acceptable, before we act on it?	Pydantic at MCP, `bootstrap --verify`, generator reference checks, `validate_nac`, OPA policy. All happen **before** the change.
Verification	Did the *network* end up as intended, after we acted?	pyATS read-back and the APIC fault delta. Both happen **after** the apply.
Evidence	What exactly has been proven, and how strongly?	An explicit level on every MCP tool (`evidence=` in code), `# EVIDENCE:` comments in Terraform, NOT-SETTABLE ids, and `Evidence-Matrix.md`.

The evidence ladder



UNIT-TESTED A **unit test** exercises one piece of code in isolation, with **fakes** or **mocks** standing in for Nautobot and the APIC. pytest finds tests by convention (`test_*.py`, `test_*`) and runs them in seconds. Each new tool needs a schema test and a dispatch test; each generator change needs a positive and a negative test. **Limit:** a fake only knows what its author assumed.

INTEGRATION-TESTED An **integration test** runs real components together, for example the MCP Server over the real protocol against a live Nautobot. This level has caught bugs that unit tests could not.

PLAN-VERIFIED `terraform plan` succeeded against the real provider and refreshed the live APIC. It proves the configuration fits the provider's schema, **not** that APIC accepts it. Count the resources a plan would create: a plan once succeeded while creating zero of the intended objects.

LIVE-VERIFIED Applied to the real APIC (or, for a read tool, run against the real service), read back, fault delta clean. **109 of 138 tools** are here; the other 29 are honestly marked unit-tested.

NOT REACHED Nothing here is proven on real switches with real traffic.

pytest vs pyATS — the two are often confused

	pytest	pyATS
What it is	The standard general-purpose Python test runner	Cisco's network test framework (testbeds, connections, parsers)
What it tests	The platform's **code**: tools, generator, scripts, reasoning layer, pipeline wiring	The **network**: does the APIC contain what the YAML says?
Needs a lab?	No. Fakes and mocks.	Yes. Connects to the real APIC.
Runs in CI?	**No.** No pipeline job invokes pytest today; it runs on developer machines.	**Yes**, as the `pyats_verify` job after `ansible_configure`.
Kind	Testing (before merge)	Verification (after deployment)

Why "the unit tests pass" is never the whole story

Most of the defects found in this platform were **silent**: an operation reported success and changed nothing. Each one was invisible to unit tests, because the fakes behaved as the author assumed and not as the real system did.

HTTP 200, nothing saved

Nautobot discards writes to undeclared Custom Fields without an error. A whole L4-L7 lab was "built" through MCP and nothing persisted. Fixed with `bootstrap_nautobot.py` and a cross-check test.

Planned, but last week's intent

The plan job read the Git copy of the YAML, one pipeline behind Nautobot. Validation checked one file, Terraform planned another. Found by `review-plan` on 18 October. Fixed, with a test.

Stored, never sent

VRF descriptions reached Nautobot and the YAML, but the Terraform module never set them. Every call succeeded. Found the same day; fixed, tested, applied.

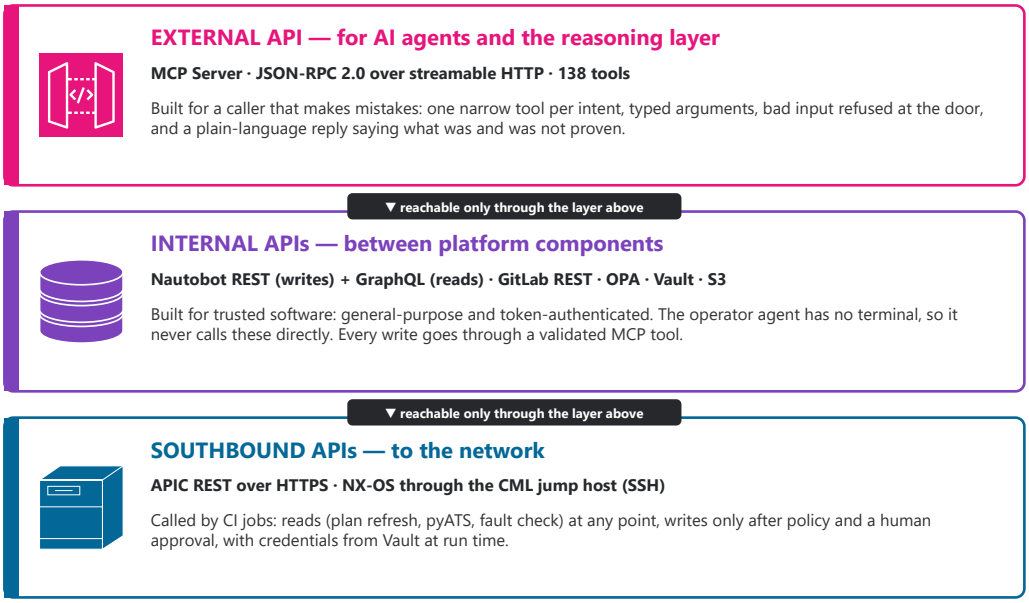
> **The rule this produced (Definition of Done, CLAUDE.md)**
>
> Every new capability ships with unit tests, *and* a live-verification attempt including an APIC fault check, *and* its evidence level declared in code, *and* the documents updated. If live verification is not possible, write down why, rather than quietly calling the capability done.

Where each check runs today

Check	Where	Automatic?
pytest (1,086 tests)	Developer machine	No. A CI job for it is a recommended next step.
MCP schema validation	MCP Server, on every tool call	Yes
Nautobot schema check	`bootstrap_nautobot.py --verify`	No. Run it before a demo and on a new site.
Reference integrity, determinism, validate, policy, plan	CI: generate → validate → policy → plan	Yes, on every pipeline
Plan review in plain words	`review-plan`, by the approver	On demand, before approving
APIC backup	CI: `terraform_apply`, before Terraform runs	Yes, every apply; fail-closed
pyATS read-back, fault delta	CI: verify stage	Yes, after every apply
Diagnosis across all four layers	`diagnose`	On demand
Drift detection	Every plan (persisted state)	**Partly**: only when a pipeline runs

8 External vs internal API design

An **API** (application programming interface) is the contract one program offers another. This platform has three layers of them, designed for three very different callers. Each layer is reachable only through the one above it.



Designing the external API (for agents)

- **One narrow tool per intent.** `create_bridge_domain`, not a generic "set any field". A narrow tool is easy for a model to choose correctly and easy to validate.
- **Thin schemas, no shared envelope (ADR-018).** Each tool has only the arguments its task needs. NetAsCode YAML, not an MCP-owned schema, is the authoritative intent format.
- **Refuse at the door.** Invalid combinations are rejected with a reason the agent can act on, before anything is written.
- **Idempotent and deduplicating.** Calling a tool twice must not create two objects.
- **Honest results.** The reply states what was stored, what was not proven, and whether the change will start a pipeline.
- **Reads for machines.** Read tools return structured JSON, so the reasoning layer can compute on them instead of parsing prose.
- **Evidence and gaps declared per tool.** `unit-tested` by default; `live-verified` must carry a note saying how; NS ids name what cannot be set.

Designing the internal APIs

- **Use each product's own API.** Nautobot REST for writes and GraphQL for reads, the GitLab trigger and state APIs, OPA's data API, Vault KV v2, S3. There are no custom services in between, and no listener for the webhook.
- **Right tool for reads and writes.** GraphQL fetches exactly the fields the generator needs in one query; REST gives simple, auditable writes.
- **Credentials at run time.** Tokens come from environment variables or Vault; nothing is committed.
- **Fail closed.** If OPA, the APIC or the backup cannot be reached, the pipeline stops rather than assuming "allowed" or "fine".

Southbound

APIC REST is called only by CI jobs and by engineers doing live verification. Writes happen only after policy, approval and a backup. The NX-OS devices are reached through the CML jump host.

Gap before a customer deployment: the MCP endpoint is unauthenticated. `MCP_API_KEY` exists in `config.py`, but no code checks it. Anyone who can reach port 8071 can call the tools, deletes included. Add a check (or an authenticating proxy) before exposing the server beyond a lab; ADR-022 also makes it a precondition for any reasoning workflow that writes.

NEW IN EDITION 2

9 LangGraph: what it is, and what it does here

LangGraph is an open-source Python library for writing a workflow as a **graph**: a set of boxes (the steps) joined by arrows (what happens next). You describe the steps and the arrows once. LangGraph then runs them in the right order, runs independent steps at the same time, retries a step that failed for a temporary reason, and carries a shared record of everything learnt so far from one step to the next.

> **An everyday picture**
>
> Imagine a **form on a clipboard** that is passed around an office. Each colleague does **one job**: one looks the tenant up in Nautobot, one fetches the generated YAML, one reads the pipeline's result. Each writes their findings into their own boxes on the form. A **wall chart** with arrows says who gets the clipboard next. Some arrows carry a **question** ("Was the tenant found?"), and the answer decides which way the clipboard goes. At the end, one colleague reads the whole form and writes the conclusion.
>
> In LangGraph's words: the form is the **state**, each colleague is a **node**, each arrow is an **edge**, an arrow with a question is a **conditional edge**, and the wall chart is the **StateGraph**. Every run begins at **START** and finishes at **END**.

Why a graph library, and not just a Python script?

- **The flow is written down, not buried in if-statements.** Anyone can read `build_graph()` and see every step and every branch, and section 9's diagram is drawn straight from it.
- **Parallel reads and retries come for free.** Two reads that do not depend on each other run at the same time; a network hiccup is retried up to three times; a real error is reported, not retried.
- **Each step can be tested on its own,** with a fake instead of the real MCP Server. The 57 reasoning tests do exactly that.
- **It is ready for what comes next.** LangGraph can pause a run, wait for a person, and resume later, even after a restart. A workflow that ever *writes* needs that, and ADR-022 makes it a precondition.

What LangGraph is not, in this project

LangGraph is often used to drive AI language models. **Here, no AI model makes any decision.** Every node is ordinary, deterministic Python, so the same inputs always give the same answer (ADR-010: AI may propose, execution must stay deterministic). LangGraph is used for its structure: steps, branches, parallelism, retries, and later, pauses for approval.

WORKFLOW 1 · DIAGNOSE

"Why is BGP_L3Out not working?"

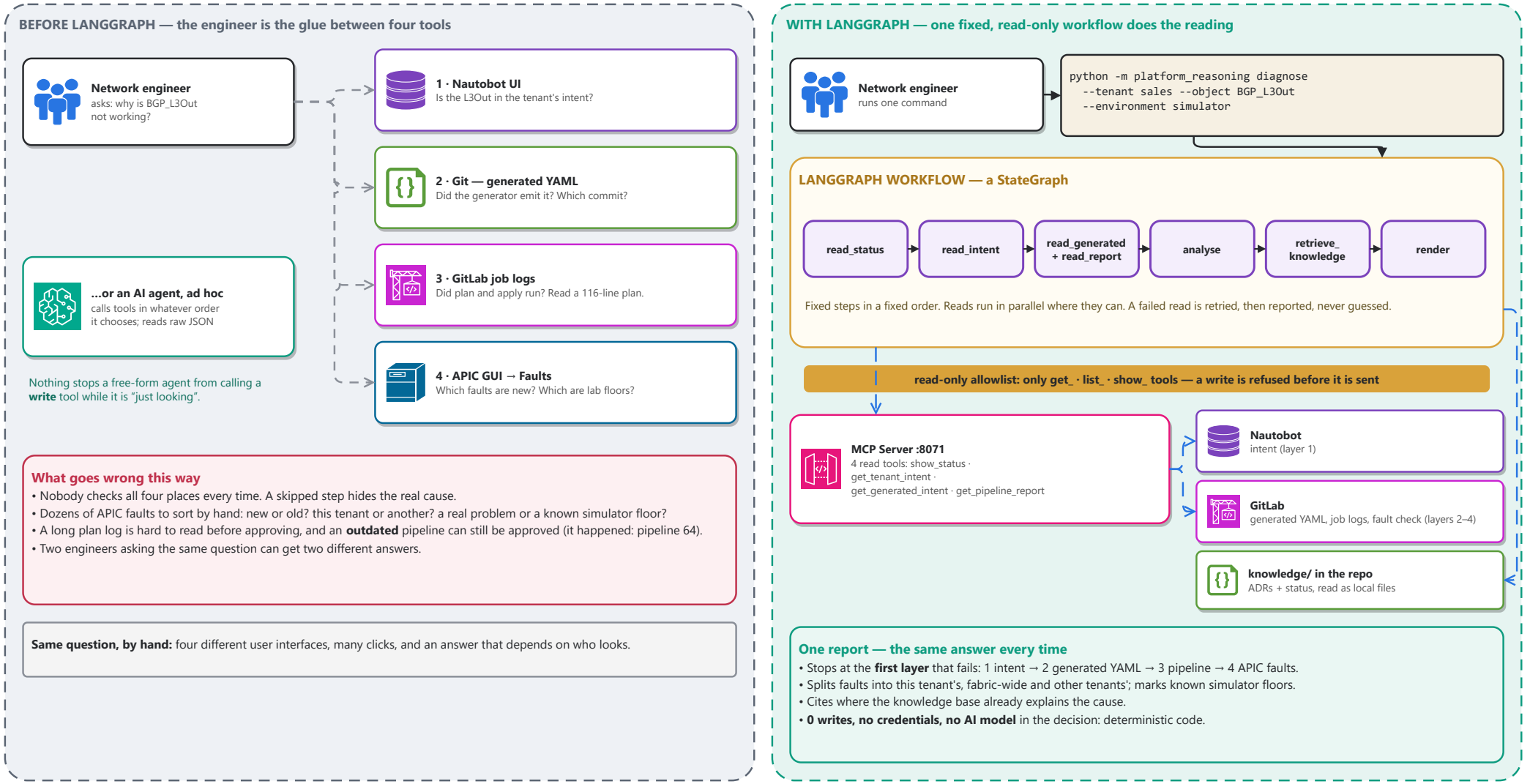
Follows one object (or a whole tenant) through four layers: Nautobot intent → generated YAML → the pipeline's Terraform result → the APIC fault check. Reports the **first layer where it stops**, and what the knowledge base already says about that cause.

WORKFLOW 2 · REVIEW-PLAN

"What will approving this pipeline change?"

Before a person approves `terraform_apply`: what the plan creates, changes, DELETES or REPLACES, by name; the caveats on those objects; drift; whether the list adds up; and an OUTDATED warning if a newer plan exists.

Where it sits: it is an MCP client like the AI agent (arrow 19 on the map), but limited to 7 read tools and holding no password. It never starts or approves a pipeline. If it is switched off, the platform works exactly as before.



The architecture did not change; the investigation did. Before, answering "why is this not working?" meant visiting four user interfaces in an order of the engineer's choosing, or letting a free-form agent do the same with no guard against writes. With LangGraph, one command runs a fixed, read-only graph that collects the same facts through the MCP Server, applies the same rules every time, and returns one report.

Side by side

	Before LangGraph	With LangGraph
Who collects the facts	The engineer, by hand, in Nautobot, Git, GitLab and the APIC GUI. Or a free-form AI agent.	Fixed nodes, through 4 MCP read tools (diagnose) or 1 (review-plan).
Order of checks	Whatever the person remembers; steps get skipped.	Always intent → YAML → pipeline → APIC faults; stops at the first failure.
APIC faults	Sorted by eye: new or old, which tenant, real or lab floor.	Split automatically: this tenant / fabric-wide / other tenants; known families named; "expected on the simulator" only with `--environment simulator`.
Before approving	Read the raw plan log (116 resources on adoption day).	A plain summary by object name, caveats, drift, an arithmetic check and an OUTDATED warning.
Safety	A free-form agent can call a write tool while "just looking".	An allowlist refuses any non-read tool before it is sent; no password held.
Consistency	Two people can reach two conclusions.	Deterministic: same inputs, same report.

Example 1 — "Why is BGP_L3Out not working?"

Before: open Nautobot and find the L3Out in tenant `sales`; open the generated `tenants.yaml` in Git and check it is there; open the pipeline's jobs and read the plan and apply logs; open the APIC GUI, list faults, and work out which of about twenty are new, which concern BGP_L3Out, and which this simulator always raises. **After:** one command. This is the real report for pipeline 60, run on 18 October:

```
Diagnosis: tenant sales, object BGP_L3Out (pipeline 60), environment production

Verdict: Stops at layer 4: APIC accepted 'BGP_L3Out' but raised 1 fault(s) on it (switch_absence), so it is
stored and not deployed. switch_absence is a lab floor, not expected outside the simulator (environment:
production) -- investigate.

  [ok  ] 1 intent (Nautobot)        BGP_L3Out found as l3out
  [ok  ] 2 generated YAML           BGP_L3Out present (artifact of job 989 (generate_nac), the file this pipeline's
Terraform consumed)
  [ok  ] 3 execution (GitLab)       plan +116 ~0 -0; applied 116 added, 0 changed, 0 destroyed
  [STOP] 4 observed (APIC faults)   14 new faults on this tenant, 1 on BGP_L3Out
```

The report goes on to show the fault itself (F1298 on external EPG Nexus_ExtNet, node 101: "Node Cannot Deploy EPG"), the passages in `knowledge/` that already explain it, and the line "*Read-only: 4 MCP call(s) ..., 0 writes.*" With `--environment simulator` the same fault is labelled an expected lab floor, because this lab has no real switch for node 101.

Example 2 — approving a pipeline

Before: on 18 October, pipeline 64 had planned an OSPF fix and was cancelled after pipeline 65 superseded it. GitLab's *Cancel* does not cancel a waiting manual job, so 64's apply was **still approvable**, and nothing on the screen said it was outdated. **After:** `review-plan --pipeline 64` starts with "*OUTDATED: pipeline 65 planned this branch after this one -- approving this would apply an outdated plan.*"

How it has improved the project

1 **An expert's method became code.**
The four-layer way of tracing a problem (is it in Nautobot? in the YAML? did the pipeline apply it? did APIC accept it as valid?) used to live in one person's head. It now lives in `analysis.py`, with tests, and anyone can run it.

2 **It found real platform bugs that every other check had missed.**
When the approval assistant was tried on a real change, its output did not match what was expected, and following that up found:

- **Every ACI plan had been one pipeline behind Nautobot.**
  The diagnosis of pipeline 67 read "*[ok] 2 generated YAML approval-demo-vrf present*" and then "*[STOP] 3 execution (GitLab) plan +0 ~0 -0*": a new VRF was in the YAML but not in the plan. Cause: the plan job read Git's copy of the YAML. Fixed for ACI and EVPN, guarded by a test.
- **VRF descriptions had never reached the APIC.**
  Pipeline 70 planned a new VRF with its description "known after apply". Fixed, applied in pipeline 74 with no new faults.
- **Superseded pipelines stayed approvable.**
  Now flagged OUTDATED, and the stale ones were deleted.

6 **Approvals are safer.**
The approver sees deletions and replacements by name, the module's own caveats, and drift, before clicking Run.

7 **Three new read tools serve everyone.**
`get_pipeline_report`, `get_generated_intent` and `get_plan_changes` also work for the Copilot agent, so it can answer "what happened?" from facts.

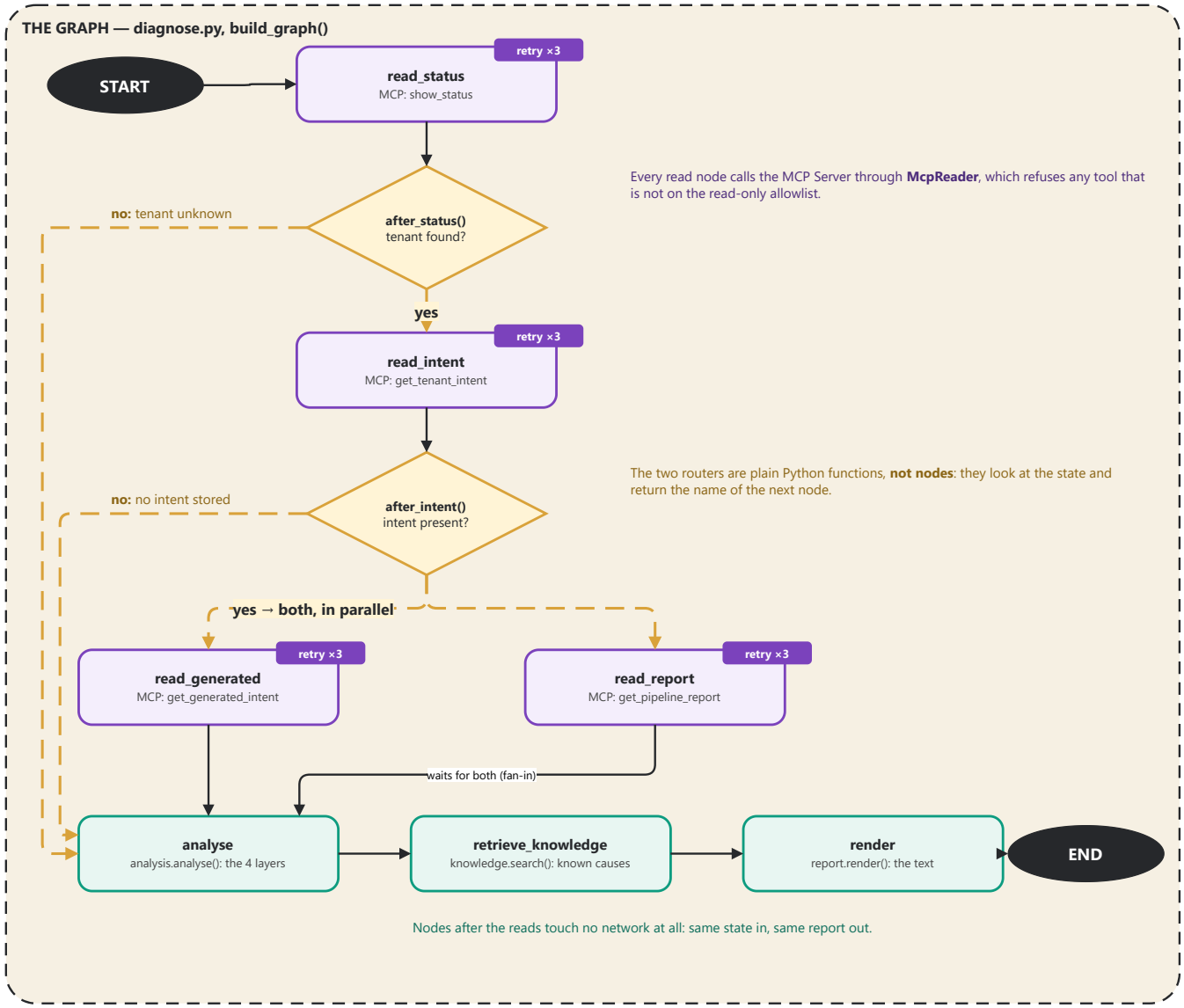
8 **The ground is prepared for guided changes.**
A future "guided build" or "safe decommission" can reuse the pattern once ADR-022's gates are met (MCP authentication, durable checkpoints, approval pauses, read-back after every write).

What it deliberately did not change

Unchanged	Why
The pipeline and its gates	LangGraph does not run inside the pipeline and cannot start or approve it. Deployments are not faster or slower.
Nautobot as the source of truth, Terraform as the executor	The reasoning layer reads both; it owns no intent and no state.
Who decides	A person approves `terraform_apply` in GitLab. The assistant's last line is always "*This review approves nothing -- the decision is yours in GitLab*".
Credentials	The reasoning process holds none. It can only ask the MCP Server read questions.

Its limits, today

- It sees the APIC only through the pipeline's own fault check, not by querying the APIC live; a live read is reserved for the future drift-report workflow.
- Knowledge retrieval is keyword search over `knowledge/`, not semantic search. ACI only: EVPN jobs are not interpreted yet.



THE STATE — class DiagnosisState(TypedDict)

One shared form that travels through the graph. Each node **reads** it and **returns only the keys it filled in**; LangGraph merges them in.

KEY	WRITTEN BY
`tenant · object_name · environment`	the input you give
`pipeline_id`	input, or read_status fills it
`status`	read_status
`intent`	read_intent
`generated`	read_generated
`pipeline_report`	read_report
`errors`	any read node (merged, not overwritten)
`diagnosis`	analyse
`knowledge`	retrieve_knowledge
`report`	render: the text you see

HOW TO READ IT

- **Rounded box = node**: one Python function, one job.
- **Solid arrow = edge**: always go there next.
- **Dashed gold arrow = conditional edge**: a router function picks the next node.
- **One arrow splitting in two = fan-out**: both nodes run at the same time; the next node waits for both.
- **START / END**: where a run enters and leaves the graph.
- **retry ×3**: a RetryPolicy. Network errors are retried; a refused write or a tool error is not.

The diagnose workflow exactly as `build_graph()` wires it. A run enters at START, reads the tenant's status, and asks "tenant found?". If not, it jumps straight to `analyse`, which reports "stops at layer 1". If found, it reads the intent and asks "intent present?". If so, it reads the generated YAML and the pipeline report **at the same time**, waits for both, then analyses, looks up known causes, and renders the text. The state on the right is the clipboard every node reads and adds to.

Every LangGraph term, and where it appears

Term	In plain words	In `diagnose` (diagnose.py)	In `review-plan` (review.py)
State schema	The shape of the shared form. A Python `TypedDict`: a list of named keys and their types. It is the **input schema of every node and every edge**: each one receives the whole state.	`DiagnosisState`, 12 keys: input (tenant, object, pipeline, environment), what each read found, the diagnosis, the knowledge, the report	`ReviewState`, 4 keys: `pipeline_id`, `plan`, `error`, `report`
StateGraph	The graph object. You create it with the state schema, then add nodes and edges to it.	`graph = StateGraph(DiagnosisState)`	`graph = StateGraph(ReviewState)`
START	A built-in marker: where every run enters.	`add_edge(START, "read_status")`	`add_edge(START, "read_plan")`
Node	One Python function. It receives the state and returns a small dict of **only the keys it filled in**; LangGraph merges that into the state.	7 nodes: `read_status`, `read_intent`, `read_generated`, `read_report` (each one MCP read), `analyse`, `retrieve_knowledge`, `render`	2 nodes: `read_plan`, `summarise`
Edge	A fixed arrow: after this node, always run that one.	`analyse → retrieve_knowledge → render → END`	`read_plan → summarise → END`
Conditional edge	An arrow with a question. A **router function** looks at the state and returns the name of the next node (or several names).	`after_status`: tenant found? · `after_intent`: intent present?	None: every plan is summarised the same way
Fan-out / fan-in	Fan-out: a router returns two names, so both nodes run at once. Fan-in: a node listed after both waits for both.	`after_intent` returns `["read_generated", "read_report"]`; `add_edge([…both…], "analyse")`	Not needed
Reducer	A rule for combining two writes to the same key, instead of the second one overwriting the first.	`errors: Annotated[dict, operator.or_]`: errors from parallel reads are merged	Not needed
RetryPolicy	Automatic retry of a failing node.	On the 4 read nodes: 3 attempts, 1 s apart, only for transient errors (`_transient`)	Same, on `read_plan`
END	A built-in marker: where a run finishes.	`add_edge("render", END)`	`add_edge("summarise", END)`
compile / ainvoke	`compile()` checks the wiring and makes it runnable; `ainvoke(initial_state)` runs it once and returns the final state.	`await graph.ainvoke({"tenant": …, "errors": {}})`	`await build_graph(reader).ainvoke({"pipeline_id": …})`

The one idea to keep

A node never edits the state directly. It **returns what it learnt**, for example `{"intent": {...}}`, and LangGraph adds it. That is why nodes can run in parallel safely, why each node can be tested with a hand-made state, and why the final state (`--json`) shows exactly what every step found.

File: `reasoning/src/platform_reasoning/diagnose.py` (150 lines). Excerpts are verbatim; "…" marks lines left out.

The state schema

```python
class DiagnosisState(TypedDict, total=False):
    tenant: str
    object_name: str | None
    pipeline_id: int | None
    environment: str
    status: dict | None
    intent: dict | None
    generated: dict | None
    pipeline_report: dict | None
    errors: Annotated[dict[str, str], operator.or_]
    diagnosis: dict
    knowledge: dict[str, list[dict]]
    report: str
```

What it does: declares the shared form. `total=False` means a key may be missing until a node fills it in. `errors` carries a **reducer**, `operator.or_`: when two parallel reads both report an error, the two dicts are merged (`{"generated": …} | {"report": …}`) instead of the second replacing the first.

Which failures are retried

```python
def _transient(exc: Exception) -> bool:
    # A refused write or a tool-level error is deterministic; retrying cannot help.
    return not isinstance(exc, (ToolCallFailed, WriteToolRefused))


_RETRY = RetryPolicy(max_attempts=3, initial_interval=1.0, retry_on=_transient)
```

What it does: a `RetryPolicy` tells LangGraph to re-run a node that raised an exception, up to 3 attempts, starting 1 second apart. The condition `_transient` limits this to temporary problems such as a dropped connection. A tool that answered with an error, or a refused write, fails the same way every time, so it is not retried.

Two of the seven nodes

```python
    async def read_intent(state: DiagnosisState) -> dict:
        try:
            return {"intent": await reader.call("get_tenant_intent", tenant=state["tenant"])}
        except ToolCallFailed as exc:
            return {"intent": None, "errors": {"intent": exc.message}}
    …
    def analyse(state: DiagnosisState) -> dict:
        diagnosis = analysis.analyse(
            tenant=state["tenant"],
            object_name=state.get("object_name"),
            intent=state.get("intent"),
            generated=state.get("generated"),
            report=state.get("pipeline_report"),
            errors=state.get("errors") or {},
            environment=state.get("environment") or "production",
        )
        return {"diagnosis": asdict(diagnosis)}
```

What they do: `read_intent` is an **async** node (it waits on the network). It reads the state, makes **one** MCP call through the read-only `reader`, and returns only `{"intent": …}`. If the tool fails it does not crash the run: it records the reason under `errors`, and `analyse` will report the layer as "unknown" with that reason. `analyse` is a plain function with no network access: it hands everything collected so far to `analysis.analyse()`, the four-layer rules shown in Part 3, and stores the result.

The two conditions (router functions)

```
    def after_status(state: DiagnosisState) -> str:
        status = state.get("status")
        return "read_intent" if status is not None and status.get("found") else "analyse"

    def after_intent(state: DiagnosisState) -> list[str] | str:
        return ["read_generated", "read_report"] if state.get("intent") else "analyse"
```

The conditions, in words. `after_status`: *if* `show_status` answered and the tenant was found, go on to `read_intent`; *otherwise* skip every other read and go to `analyse`, which will say "stops at layer 1: Nautobot holds no tenant". `after_intent`: *if* the tenant has intent, return **two** node names, so both reads run in parallel; *otherwise* go to `analyse`. Skipping reads that cannot help is why an unknown tenant costs one MCP call, not four.

Building the StateGraph

```
    graph = StateGraph(DiagnosisState)
    for name, node in (
        ("read_status", read_status),
        ("read_intent", read_intent),
        ("read_generated", read_generated),
        ("read_report", read_report),
    ):
        graph.add_node(name, node, retry_policy=_RETRY)
    graph.add_node("analyse", analyse)
    graph.add_node("retrieve_knowledge", retrieve_knowledge)
    graph.add_node("render", render)

    graph.add_edge(START, "read_status")
    graph.add_conditional_edges("read_status", after_status, ["read_intent", "analyse"])
    graph.add_conditional_edges("read_intent", after_intent, ["read_generated", "read_report", "analyse"])
    graph.add_edge(["read_generated", "read_report"], "analyse")
    graph.add_edge("analyse", "retrieve_knowledge")
    graph.add_edge("retrieve_knowledge", "render")
    graph.add_edge("render", END)
    return graph.compile()
```

What it does, line by line. Create the graph with the state schema. Add the four read nodes *with* the retry policy and the three local nodes without it (they cannot fail transiently). `add_edge(START, …)` sets the entry. `add_conditional_edges(node, router, targets)` attaches a router to a node; the list names every node the router may return, so LangGraph can check the wiring at `compile()`. `add_edge([a, b], "analyse")` is the **fan-in**: `analyse` waits for both reads. The rest is a straight line to END.

Running it

```
async def diagnose(reader, tenant, object_name=None, pipeline_id=None, sections=None,
                   environment="production") -> DiagnosisState:
    graph = build_graph(reader, sections if sections is not None else knowledge.load_sections())
    return await graph.ainvoke(
        {"tenant": tenant, "object_name": object_name, "pipeline_id": pipeline_id,
         "environment": environment, "errors": {}}
    )
```

What it does: builds the graph around an MCP reader and the loaded knowledge base, then runs it once with the initial state (what you typed on the command line). `ainvoke` returns the final state; the command prints its `report` key, or the whole state with `--json`. (Signature shortened here; the file carries type hints.)

The read-only allowlist (mcp_reader.py)

```
READ_ONLY_TOOLS: frozenset[str] = frozenset({
    "show_status", "list_aci_tenants", "get_tenant_intent", "get_fabric_intent",
    "get_generated_intent", "get_pipeline_report", "get_plan_changes",
})

class McpReader:
    """Calls read-only MCP tools and returns their JSON payload as a dict."""

    async def call(self, tool: str, **arguments: Any) -> dict:
        if tool not in READ_ONLY_TOOLS:
            raise WriteToolRefused(f"'{tool}' is not a read-only tool; this workflow never writes")
        self.calls.append(tool)
        result = await self._session.call_tool(tool, {k: v for k, v in arguments.items() if v is not None})
        if getattr(result, "is_error", False):
            raise ToolCallFailed(tool, _text(result) or "tool reported an error")
        return _payload(result)
```

What it does: every node's network access goes through `McpReader.call`. The **first line** refuses any tool not on the list, before a request leaves the process, so a mistake in a node (or a future AI-model node proposing a write) cannot change anything. A test also checks that every allowed name starts with `get_`, `list_` or `show_`. `self.calls` records each call for the report's last line: "*Read-only: 4 MCP call(s) …, 0 writes*". At connect time, `connect()` checks that the server really offers the tools this workflow needs.

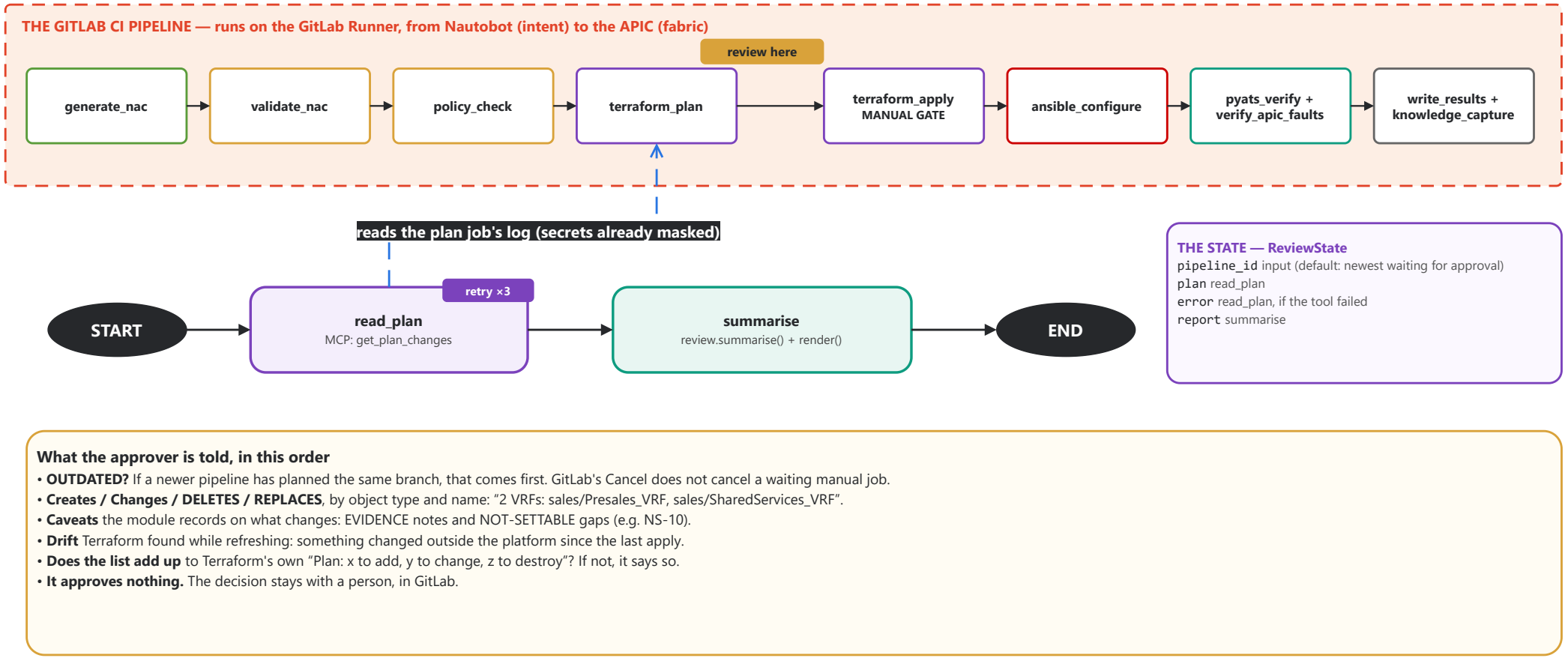
The four-layer rules (analysis.py, layer 3 shown)

```
    plan = _terraform(report, ACI_JOBS["plan"])
    apply = _terraform(report, ACI_JOBS["apply"])
    …
    if _awaiting(report, ACI_JOBS["apply"]):
        d.layers.append(Layer(3, "execution (GitLab)", "stop", f"{plan_text}; terraform_apply awaits manual approva
        d.verdict = f"Stops at layer 3: pipeline {pipeline_id} planned the change but nobody has approved terraform
        return d
    if apply is None:
        …  # "Stops at layer 3: pipeline … never ran terraform_apply."
    if apply.get("status") != "success" or apply.get("errors"):
        …  # "Stops at layer 3: terraform_apply failed in pipeline … -- <first error>"
```

What it does: `analyse()` walks the layers in order and returns at the **first** one that stops, each with a plain verdict. Layer 1: is the tenant or object in Nautobot (with "did you mean…?" suggestions)? Layer 2: is it in the YAML this pipeline's Terraform consumed? Layer 3 (above): was the apply approved, did it run, did it succeed? Layer 4: did APIC raise new faults, on this tenant or on this object, and do they belong to a **known family** (switch absence, L4-L7 without a concrete device, route-reflector node not a spine, vPC pair not in the fabric)? Job names are matched exactly: `ACI_JOBS` exists because a suffix match once confused `evpn_terraform_apply` with the ACI job.

The Python classes and functions at a glance

Name	Kind	Role
`DiagnosisState`, `ReviewState`	TypedDict	The state schemas of the two graphs
`build_graph()`, `diagnose()`, `review()`	functions	Wire and compile a graph; run it with `ainvoke`
`McpReader`, `connect()`	class, async context manager	Read-only MCP access; opens the session and checks the required tools
`WriteToolRefused`, `ToolCallFailed`	exception classes	A refused write; a tool that answered with an error (neither is retried)
`analysis.analyse()`; `Layer`, `Diagnosis`, `FaultFamily`	function; dataclasses	The four-layer rules and their results; known fault families with the environments they are expected in
`knowledge.load_sections()`, `search()`	functions	Split `knowledge/*.md` into sections; keyword search for known causes
`report.render()`; `review.summarise()`, `PlanSummary`	functions; dataclass	Turn the final state into the text you read



What the approver is told, in this order

- **OUTDATED?** If a newer pipeline has planned the same branch, that comes first. GitLab's Cancel does not cancel a waiting manual job.
- **Creates / Changes / DELETES / REPLACES**, by object type and name: "2 VRFs: sales/Presales_VRF, sales/SharedServices_VRF".
- **Caveats** the module records on what changes: EVIDENCE notes and NOT-SETTABLE gaps (e.g. NS-10).
- **Drift** Terraform found while refreshing: something changed outside the platform since the last apply.
- **Does the list add up** to Terraform's own "Plan: x to add, y to change, z to destroy"? If not, it says so.
- **It approves nothing.** The decision stays with a person, in GitLab.

Where it sits. Between `terraform_plan` and the manual gate on `terraform_apply`. It is not a pipeline job: the approver runs it, and it reads the plan job's log through one MCP tool, `get_plan_changes`. That log has already passed through `mask_secrets.py`, so no SNMP community string or password appears in it.

Why the graph is so short. Two nodes are enough: one read, one summary. It is still a LangGraph graph so it shares the retry policy, the read-only reader, the testing style and the future pause-for-approval mechanism with `diagnose`. Without a pipeline id, it picks the newest pipeline whose `terraform_apply` is waiting.

The graph (review.py)

```
class ReviewState(TypedDict, total=False):
    pipeline_id: int | None
    plan: dict
    error: str
    report: str


def build_graph(reader: Any):
    async def read_plan(state: ReviewState) -> dict:
        try:
            return {"plan": await reader.call("get_plan_changes", pipeline_id=state.get("pipeline_id"))}
        except ToolCallFailed as exc:
            return {"plan": {"found": False, "reason": f"get_plan_changes failed: {exc.message}"}, "error": exc.mes

    def summarise_node(state: ReviewState) -> dict:
        return {"report": render(summarise(state["plan"]), reads=list(getattr(reader, "calls", ["get_plan_changes"]

    graph = StateGraph(ReviewState)
    graph.add_node("read_plan", read_plan, retry_policy=RetryPolicy(max_attempts=3, initial_interval=1.0, retry_on=
    graph.add_node("summarise", summarise_node)
    graph.add_edge(START, "read_plan")
    graph.add_edge("read_plan", "summarise")
    graph.add_edge("summarise", END)
    return graph.compile()
```

The conditions inside summarise()

```
    if not payload.get("consistent"):
        headline = (f"WARNING: the parsed list does not add up to Terraform's own Plan line (plan {summary}, "
                    f"parsed {counted}) -- read the job log before approving.")
        risk = "incomplete"
    elif not changes:
        headline, risk = "No changes: approving would apply nothing.", "low"
    else:
        …  # "Creates N, changes N, DELETES N[, REPLACES N]." + "Review each deletion …" when any
    # An outdated plan outranks everything else: GitLab's cancel does not
    # cancel a waiting manual job, so an older pipeline stays approvable.
    if payload.get("superseded_by"):
        headline = (f"OUTDATED: pipeline {payload['superseded_by']} planned this branch after this one -- "
                    f"approving this would apply an outdated plan. …")
        risk = "outdated"
```

In words: if the parsed resource list does not add up to Terraform's own "Plan:" line, say so and send the approver to the log; never present an incomplete list as complete. If there are no changes, say approving would do nothing. Otherwise count creations, changes, deletions and replacements, and call out every deletion. Finally, if a newer pipeline has planned the same branch, put **OUTDATED** first, whatever the plan says.

The real output for pipeline 74 (19 October)

```
Plan review: pipeline 74 (branch copilot/aci-platform-comparison, commit 6167fab3) -- terraform_apply is WAITING FOR
YOUR APPROVAL

Creates 0, changes 2, deletes 0. Nothing is deleted or replaced.

Changes:
  2 VRFs: sales/Presales_VRF, sales/SharedServices_VRF

Caveats the module records on what changes:
  VRF: NS-10 not yet settable -- Further VRF / BD / EPG attributes: e.g. BD IP learning and limit-learning-to-
subnets, ...

Check: The list adds up to Terraform's own Plan line (0 to add, 2 to change, 0 to destroy).

Before Terraform runs, the apply job takes an APIC configuration backup (apic-backup.json).

This review approves nothing -- the decision is yours in GitLab: http://localhost:8929/root/nautobot-infra-
automation/-/pipelines/74
Read-only: 1 MCP call(s) (get_plan_changes), 0 writes.
```

The user then approved pipeline 74: backup taken, "*Apply complete! Resources: 0 added, 2 changed, 0 destroyed*", no new APIC faults (23 before, 23 after).

Run it (Windows PowerShell, from the repository root)

```
cd reasoning
$env:PYTHONPATH = 'src'
# this lab is the simulator: say so, or its known fault floors stop every diagnosis
$env:PLATFORM_ENVIRONMENT = 'simulator'

# Why is an object not (validly) on the APIC?
.venv/Scripts/python -m platform_reasoning diagnose --tenant sales --object BGP_L3Out
.venv/Scripts/python -m platform_reasoning diagnose --tenant sales            # the whole tenant
.venv/Scripts/python -m platform_reasoning diagnose --tenant sales --pipeline 74

# What will approving change?  (default: the newest pipeline waiting for approval)
.venv/Scripts/python -m platform_reasoning review-plan
.venv/Scripts/python -m platform_reasoning review-plan --pipeline 74 --json   # full state as JSON
```

Needs: the MCP Server container running on `http://localhost:8071/mcp` (override with `--mcp-url` or `MCP_URL`). Nothing else: no Nautobot token, no GitLab token, no APIC password. If the server lacks a required tool, the command stops with "*MCP Server … lacks […]; rebuild docker/mcp-server*".

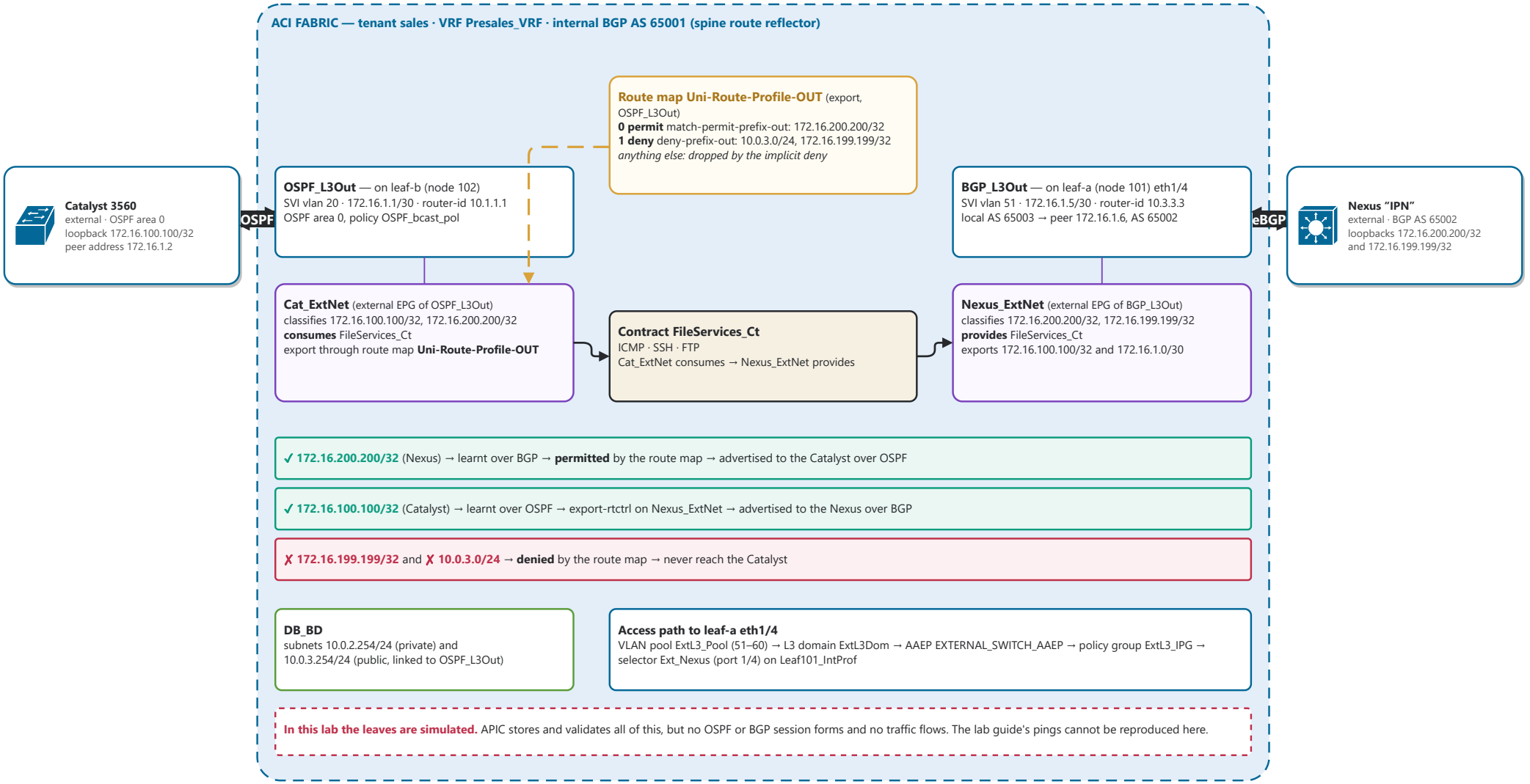
Reading a diagnosis

Marker	Meaning
`[ok ]`	This layer has what it should.
`[STOP]`	The first layer where the object stops. The verdict explains why, and "Next:" suggests what to do.
`[warn]`	Not blocking, but worth a look (for example, faults that are known simulator floors).
`[ ?? ]` / unknown	The layer could not be read (a tool failed); the reason is shown, never guessed.

File map

File	Contents
`reasoning/src/platform_reasoning/diagnose.py`	The diagnose graph: state, nodes, routers, wiring
`…/review.py`	The review-plan graph and its plain-words summary
`…/analysis.py`	The four-layer rules and the known fault families; no I/O
`…/mcp_reader.py`	The read-only MCP client and its allowlist
`…/knowledge.py`, `…/report.py`	Keyword search over `knowledge/`; the text report
`…/__main__.py`	The command line: `diagnose` and `review-plan`
`reasoning/tests/`	57 offline tests, each graph run against a fake reader
`mcp-server/…/tools/aci_lifecycle.py`, `generic.py`	The three read tools the layer relies on
`knowledge/adr/ADR-022-LangGraph-Reasoning-Layer.md`	The decision, the gates before any write workflow, the evidence

> **Run the tests yourself**
>
> `cd reasoning; $env:PYTHONPATH='src'; .venv/Scripts/python -m pytest -q` should report 57 passed. They need no lab: each graph runs against a fake MCP reader built from recorded payloads.



What the lab builds. The ACI fabric becomes a **transit** between two external routers: a Catalyst that speaks OSPF to leaf-b, and a Nexus ("IPN") that speaks eBGP to leaf-a. Routes learnt from one side are re-advertised to the other, under control: a **route map** decides what the Catalyst may learn (172.16.200.200/32 yes; 172.16.199.199/32 and 10.0.3.0/24 no), and a **contract** decides what traffic may cross (ICMP, SSH, FTP). It follows *Enable Transit Routing (DCACIA lab)*, using a route map instead of per-subnet export flags. It was first built through MCP on 16 September 2026 and lives in tenant `sales`.

10 Running the transit-routing lab

This section walks through the lab as you would run it: what you type, which element does what, and what passes between them. Section 3 told the story of one change in general; this is the same machinery, concrete.

The elements you will meet, in this lab

Element	What it is	Its job in this lab
AI agent	GitHub Copilot Chat in VS Code, in Agent mode, persona *Network Platform Operator*	Turns each of your prompts into MCP tool calls, in the right order
MCP Server	The platform's tool server, `localhost:8071`	Validates each call and stores the intent in Nautobot
Nautobot	The source of truth, `localhost:8080`	Holds the lab as data: the L3Out, route map and contracts on Tenant `sales`; the access policies on the fabric Location; the bridge-domain subnet as a Prefix
Webhook	Nautobot's change notification, sent by its Celery worker	Starts the pipeline when tenant `sales` changes
GitLab	Git repository + CI server, `localhost:8929`	Holds the pipeline definition and every rendered YAML; schedules the jobs; holds Terraform's state; shows the approval button
GitLab Runner	The worker that runs each job in a fresh container	Runs generate, validate, policy, plan, apply, configure, verify, capture
Generator	Python in `platform/python/generator/`, run by `generate_nac`	Reads all intent from Nautobot (GraphQL) and **renders** it
Rendered YAML	`platform/netascode/aci/tenants.yaml`	The lab as a file: BGP_L3Out, Nexus_ExtNet, Uni-Route-Profile-OUT… This is what Terraform reads
Terraform	The IaC engine with the CiscoDevNet/aci provider	Compares the YAML with its state and the live APIC; after approval, makes the APIC match
ACI / APIC	The fabric controller (here a simulator, `172.30.46.103`)	Stores and validates the configuration; raises faults on anything invalid

Checklist before the first prompt

1. **Docker Desktop is running and the lab containers are up.**
   `docker ps` should list Nautobot (and its **celery worker**), GitLab and its runner, Vault, OPA, MinIO and `mcp-server`. Never use `docker compose down` in a sub-folder (Hazard 1 in `Platform-Status-and-Pending-Items.md`).

2. **The Nautobot schema is complete.**
   `python platform/workflows/scripts/bootstrap_nautobot.py --verify` → `OK: schema complete` (40 fields).

3. **The APIC answers.**
   `curl -sk -o NUL -w "%{http_code}" https://172.30.46.103/` → `200`. If not, check `docker network ls` for a 172.30.x collision before assuming an outage.

4. **VS Code sees the tools.**
   Open the repository, run *MCP: List Servers*, start `nautobot-aci-platform`, open Copilot Chat (Ctrl+Alt+I), switch to **Agent** mode and pick **Network Platform Operator** in the agent picker.

How to use this page: send the prompts **one at a time**, read each reply, and answer the agent's questions. Each prompt lists the tools the agent should call, where the intent lands in Nautobot, and whether that write starts a pipeline by itself. The lab's starting point (OSPF_L3Out, Cat_ExtNet, contract FileServices_Ct, the Catalyst and Nexus configuration) already exists, as in the course lab.

0 Look before you change anything

"List the ACI tenants, then show me the current intent of tenant sales: its VRFs, bridge domains, L3Outs and contracts."

Tools: `list_aci_tenants`, `get_tenant_intent` (read only) · Nothing is written.

A The access path to leaf-a eth1/4 (fabric-wide intent)

"Create a static VLAN pool ExtL3_Pool with VLANs 51 to 60."

Tool: `create_vlan_pool` LIVE · Lands in: fabric Location, `aci_fabric_policies`

"Create a Layer 3 domain ExtL3Dom that uses VLAN pool ExtL3_Pool."

Tool: `create_l3_domain` LIVE · Location

"Create an AAEP EXTERNAL_SWITCH_AAEP attached to the L3 domain ExtL3Dom."

Tool: `create_aep` (domain type `l3`) LIVE · Location

"Create a leaf access-port policy group ExtL3_IPG that uses AAEP EXTERNAL_SWITCH_AAEP."

Tool: `create_leaf_interface_policy_group` LIVE · Location

"On access port profile Leaf101_IntProf, add a port selector Ext_Nexus for port 1/4 using policy group ExtL3_IPG."

Tools: `create_access_port_selector`, `create_access_port_block` UNIT · Location

Stage A writes to a **Location**, which does not fire the webhook. The intent waits for the next pipeline (stage B will cause one).

B The BGP L3Out towards the Nexus (tenant intent)

"In tenant sales, create a BGP L3Out named BGP_L3Out in VRF Presales_VRF, using L3 domain ExtL3Dom, with an external EPG Nexus_ExtNet that classifies 172.16.200.200/32."

Tool: `create_bgp_l3out` UNIT · Lands in: Tenant `sales`, `aci_l3outs` · **Fires the webhook**

"Add a node profile L101 to BGP_L3Out for node 101 in pod 1, router ID 10.3.3.3, not used as a loopback."

Tool: `create_l3out_node_profile` UNIT · Tenant

"Add an interface profile BGP_L3Out_interfaceProfile to that node profile, with an SVI on node 101 port 1/4, VLAN 51, address 172.16.1.5/30."

Tools: `create_l3out_interface_profile`, `create_l3out_interface` UNIT · Tenant

"On that interface, add a BGP peer 172.16.1.6 with remote AS 65002 and local AS 65003."

Tool: `create_bgp_peer` UNIT · Tenant

Read this before stage B. These five tools are **unit-tested only**: the lab's BGP_L3Out was imported from a workbook, so they have never been driven over MCP against the live lab. Expect possible surprises, and read the plan review carefully. Also, the persona's instructions still list only seven example tools under "ONLY use"; it can reach all 138, but if it hesitates, say "*use the nautobot-aci-platform MCP tools for this*".

C The route map that controls what the Catalyst learns

"In tenant sales, create a match rule match-permit-prefix-out for 172.16.200.200/32, and a match rule deny-prefix-out for 10.0.3.0/24 and 172.16.199.199/32."

Tools: `create_match_rule` (and `add_match_rule_prefix` for the second prefix) **LIVE** · Tenant, `aci_route_control` · Fires the webhook

"On OSPF_L3Out, create a route map Uni-Route-Profile-OUT of type 'match routing policy only', with entry 0 permit using match-permit-prefix-out and entry 1 deny using deny-prefix-out."

Tool: `create_route_control_profile` (type `global`) **LIVE** · Tenant, inside `aci_l3outs`

"Apply Uni-Route-Profile-OUT in the export direction on external EPG Cat_ExtNet."

Tool: `bind_external_epg_route_control_profile` **LIVE** · Tenant. Without this binding a custom-named map does nothing, and the tool warns you.

D Subnets: what each side may learn, and the bridge domain

"On BGP_L3Out's external EPG Nexus_ExtNet, add 172.16.199.199/32 as an external subnet for the EPG, and add 172.16.100.100/32 and 172.16.1.0/30 with export route control only."

Tool: `add_external_epg_subnet` ×3 **LIVE** · Tenant

"On Cat_ExtNet, change 172.16.200.200/32 to import-security only."

Tool: `set_external_epg_subnet_scope` **LIVE** · Tenant. The route map, not a subnet flag, now controls its export.

"Add a second subnet 10.0.3.254/24 to bridge domain DB_BD in VRF Presales_VRF, public, advertised through OSPF_L3Out."

Tool: `create_bridge_domain` (`subnet_scope`, `l3outs`) **LIVE** · Lands in: a Nautobot **Prefix** · Does not fire the webhook (a tenant change will)

E The contract that lets traffic cross

"Make external EPG Nexus_ExtNet provide the contract FileServices_Ct. Cat_ExtNet already consumes it."

Tool: `bind_external_epg_contract` **LIVE** · Tenant · Fires the webhook

F Follow it through

"Did a pipeline start for tenant sales? What is its status?"

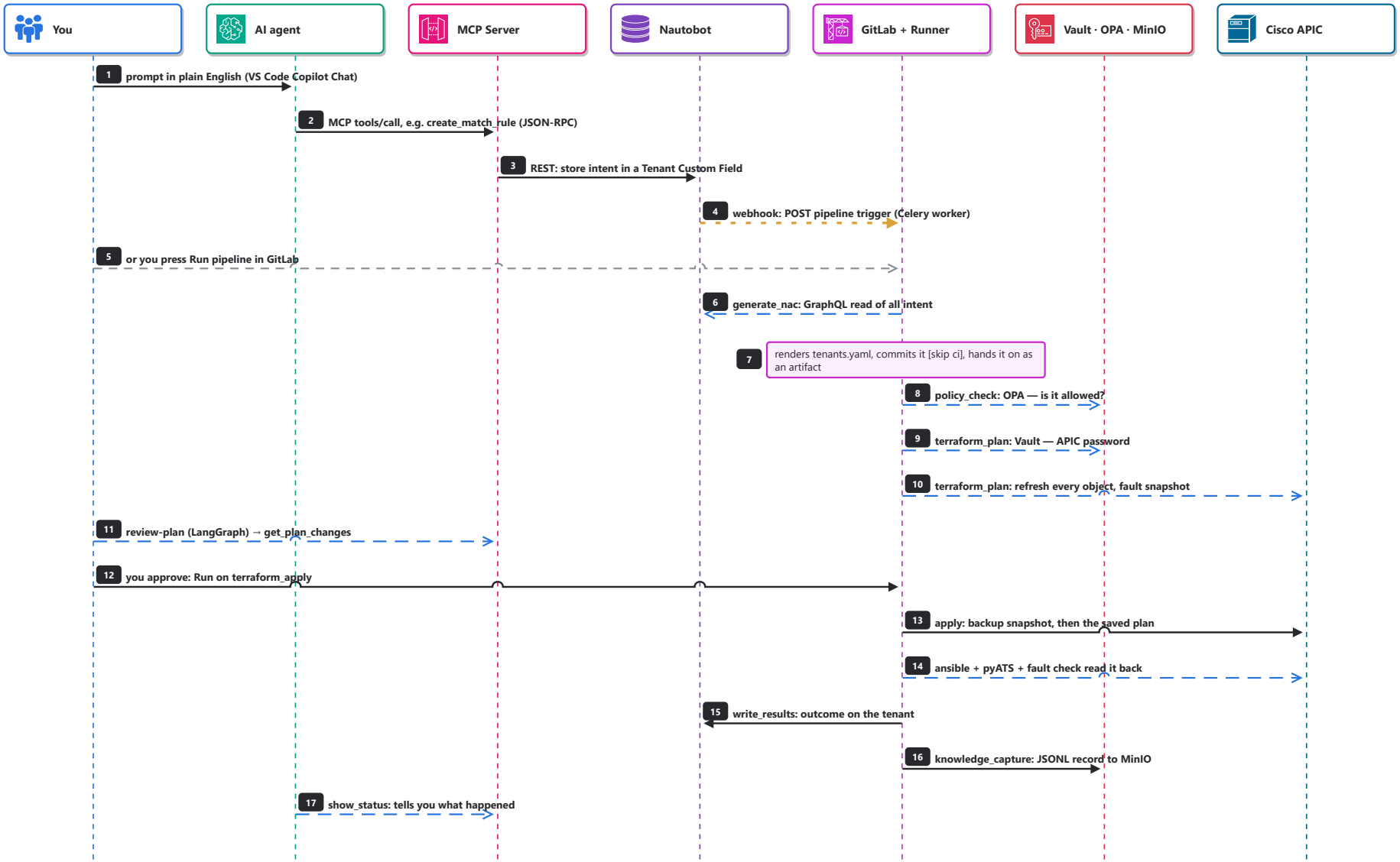
Tool: `show_status` (read only). Then run `review-plan` yourself (step 11 in the element-by-element table) before approving in GitLab.

Today, start the pipeline yourself

Two things stop the webhook from doing it in this lab right now. The Nautobot **Celery worker** that sends webhooks is down. And the webhook triggers branch `main`, which, at the last check, lagged the work branch `copilot/aci-platform-comparison` and lacked this month's fixes (backup, persisted state, the YAML fix). Until both are sorted out: GitLab → *root/nautobot-infra-automation* → *Build › Pipelines* → **Run pipeline** → choose branch `copilot/aci-platform-comparison` → **Run pipeline**. One run after the last prompt deploys everything.

Running it again on today's lab

These objects already exist in tenant `sales`. Most tools refuse a duplicate name, replace an entry of the same name, or skip an identical one, so a correct re-run should end in a plan of **"No changes"**: a good demonstration of idempotence. If the plan review shows anything you did not expect, do not approve it.



Read it top to bottom. Each column is one element; each arrow is one conversation between two of them, numbered in time order. Solid arrows change something, blue dashed arrows only read, the gold arrow is the webhook event, and the grey arrow is the manual start you use today. The next two pages explain every number.

#	Between	What happens, and what travels	Where you can see it
1	You → AI agent	Your sentence. The agent already holds the list of 138 tools and their argument schemas (MCP `tools/list`), and decides which to call. It may ask you a question first.	Copilot Chat
2	AI agent → MCP Server	A JSON-RPC `tools/call`, e.g. `create_match_rule` with `{"tenant": "sales", "name": "deny-prefix-out", "prefixes": […]}`. Pydantic checks it; a bad call is refused with a reason and nothing is written.	The tool call and its reply, expandable in the chat
3	MCP Server → Nautobot	A REST `PATCH` that merges the new entry into a JSON Custom Field: `aci_route_control` or `aci_l3outs` on Tenant `sales`, `aci_fabric_policies` on the Location, or a Prefix for the bridge-domain subnet. The reply says what was stored and whether this write will start a pipeline.	Nautobot → Tenants → *sales* → Custom Fields
4	Nautobot → GitLab	The **webhook**: Nautobot's Celery worker POSTs to GitLab's pipeline-trigger API with a trigger token. GitLab creates a pipeline with source `trigger`.	GitLab → Build › Pipelines
5	You → GitLab	Today's route: **Run pipeline** on `copilot/aci-platform-comparison`. Same pipeline, source `web`.	GitLab → Build › Pipelines
6	GitLab Runner (`generate_nac`) → Nautobot	The Runner, which polls GitLab for work, starts `generate_nac` in a fresh container. The **generator** sends one GraphQL query for all ACI intent and checks every reference. A route-map entry naming a missing match rule stops the pipeline here, with a readable error.	Job log of `generate_nac`
7	inside `generate_nac` → Git	The **rendered YAML**: `tenants.yaml` is written twice and compared (determinism), committed with `[skip ci]`, and published as an **artifact**. Every later job downloads this artifact, not the Git copy.	The commit in the repository; the job's artifacts
8	Runner (`policy_check`) → OPA	`validate_nac` has checked the YAML's shape. `policy_check` sends the intent to OPA and gets allow or deny with reasons (naming rules, protected tenants). If OPA is unreachable the answer is no.	Job logs of `validate_nac`, `policy_check`
9	Runner (`terraform_plan`) → Vault	Reads the APIC username and password and exports them as `TF_VAR_*` for this job only.	Job log: `[vault] Exported …`
10	Runner (`terraform_plan`) → GitLab state + APIC	**Terraform** locks and reads its state `cisco_aci-lab` from GitLab, refreshes every object it manages from the APIC (read-only), compares the result with the YAML, and saves the exact list of changes as `tfplan`. Then it snapshots current APIC faults. For this lab you would expect new `aci_l3_outside`, `aci_bgp_peer_connectivity_profile`, `aci_route_control_profile`, `aci_match_rule` and similar resources, or "No changes" on a re-run.	Job log of `terraform_plan`: the "Plan: x to add…" line

#	Between	What happens, and what travels	Where you can see it
11	You (`review-plan`) → MCP Server → GitLab	The **LangGraph** approval assistant reads the plan job's log through `get_plan_changes` and prints the summary: what is created, changed, deleted or replaced, the caveats (e.g. NS-28, L3Out routing settings not settable yet), drift, the arithmetic check, and OUTDATED if a newer pipeline exists.	Your terminal
12	You → GitLab	**The approval.** Open the pipeline; in the *apply* stage, `terraform_apply` shows a ▶ play button. Press it and confirm **Run job**. Nothing reaches the APIC until you do.	The pipeline page
13	Runner (`terraform_apply`) → APIC	First the **backup**: `apic_backup.py` takes a configuration snapshot and waits until APIC reports it complete. Then Terraform applies **the saved plan**, as REST calls to managed objects such as `uni/tn-sales/out-BGP_L3Out` , and saves the new state to GitLab. If anything changed since the plan, Terraform refuses the stale plan.	Job log: `OK: snapshot … taken` , then `Apply complete!`
14	Runner (`ansible_configure` , `pyats_verify` , `verify_apic_faults`) → APIC	Day-2 tasks, then two independent checks: pyATS reads the tenant and VRFs back; the fault check compares APIC faults with the snapshot from step 10 and fails on anything new on a managed object.	Job logs; `OK: no new configuration faults …`
15	Runner (`write_results`) → Nautobot	Writes the outcome (pipeline id, status) onto Tenant `sales` , so the source of truth shows whether its intent deployed.	Nautobot: the tenant's status fields
16	Runner (`knowledge_capture`) → MinIO	Appends one JSONL record: intent, commit, pipeline, outcome.	MinIO bucket `knowledge-capture`
17	AI agent → MCP Server	Ask the agent for the status: `show_status` merges Nautobot's recorded result with GitLab's live pipeline status and the agent tells you in plain words.	Copilot Chat

Check the result in the APIC GUI

- **Tenants › sales › Networking › L3Outs › BGP_L3Out**: the node profile (node 101, router ID 10.3.3.3), the SVI on eth1/4 VLAN 51, the BGP peer 172.16.1.6, and the external EPG Nexus_ExtNet with its four subnets and the provided contract.
- **Tenants › sales › Networking › L3Outs › OSPF_L3Out › Route map for import and export route control**: Uni-Route-Profile-OUT with its two entries; **Tenants › sales › Policies › Protocol › Match Rules**: the two match rules.
- **Fabric › Access Policies**: ExtL3_Pool, ExtL3Dom, EXTERNAL_SWITCH_AAEP, ExtL3_IPG, and selector Ext_Nexus on Leaf101_IntProf.

What this lab can and cannot show. The APIC is a simulator with no real leaves. It stores and validates everything above, and the relations to switch ports sit at `unformed` . No OSPF or BGP session forms, no route is exchanged, and the lab guide's `ping` and `show ip route` steps cannot be reproduced. Expect the known switch-absence fault on Nexus_ExtNet (F1298, node 101). If anything else looks wrong, run `diagnose --tenant sales --object BGP_L3Out` and read the first `[STOP]` .

Glossary

Term	Meaning here
ADR	Architecture Decision Record: a short document recording one decision and why. `knowledge/adr/`.
Artifact	A file a CI job produces and passes on (YAML, tfplan, backup record, reports, JSONL). Passed only to jobs that list the producer in `needs:`.
Custom Field	A user-defined field on a Nautobot object. The platform stores ACI intent in 40 `aci_*` fields.
Determinism	Same input, same output, every time. Makes Git diffs and diagnoses trustworthy.
DN / MO	Distinguished Name / Managed Object: the unique address and the unit of configuration in the APIC.
Drift	A difference between desired and observed state that the platform did not cause.
Node / edge / conditional edge	LangGraph: a node is one Python function (one step); an edge says which node runs next; a conditional edge lets a small router function choose.
Fail-closed	When a check cannot run, treat the result as "no".
Fault delta	Faults present after a change that were not present before it, on objects the platform manages.
Idempotent	Doing it twice has the same effect as doing it once.
LangGraph	An open-source Python library for writing workflows as graphs of steps over a shared state. Used here, without any AI model, for the read-only reasoning layer.
MCP / JSON-RPC 2.0	Model Context Protocol, an open standard for exposing tools to AI models; built on JSON-RPC, a small request/response format.
NetAsCode YAML	The rendered intent file (`tenants.yaml`) Terraform consumes; the authoritative intent artifact (ADR-018).
NOT-SETTABLE (NS-xx)	A setting the APIC supports but the platform cannot set yet, named so its absence is never silent.
Route map (route-control profile)	An ordered list of permit/deny entries deciding which routes an L3Out advertises; ends in an implicit deny.
Runner	The GitLab worker that pulls and executes CI jobs.
State / StateGraph	LangGraph: the shared record nodes read and add to (its shape, a TypedDict, is the state schema); the graph object built around it, entered at START and left at END.
State (Terraform)	Terraform's record linking each declared resource to a real object; kept in GitLab for the ACI pipeline.
Transit routing	Using the ACI fabric to pass routes, and traffic, between two external networks.
Webhook	An HTTP call one system makes to another when an event happens.

TRY THIS TODAY

1. **Check the source of truth is complete:** `python platform/workflows/scripts/bootstrap_nautobot.py --verify`.
2. **Run all three test suites:** `python -m pytest tests/unit -q` ; in `mcp-server` , `PYTHONPATH=src python -m pytest tests -q` ; in `reasoning` , `.venv/Scripts/python -m pytest -q` .
3. **Ask the reasoning layer a question:** `python -m platform_reasoning diagnose --tenant sales --environment simulator` .
4. **Before your next approval:** `python -m platform_reasoning review-plan` .
5. **Open the diagrams in draw.io:** every figure has an editable `.drawio` file next to this PDF.

Sources: this repository at commit 3cfb51d (19 Oct 2026): `pipelines/` , `mcp-server/src/mcp_server/` , `reasoning/src/platform_reasoning/` , `platform/python/generator/` , `platform/terraform/aci/` , `platform/workflows/scripts/` , `tests/` , `platform/netascode/aci/tenants.yaml` ; ADR-014/016/018/019/020/021/022; `Platform-Status-and-Pending-Items.md` , `Evidence-Matrix.md` , `Platform-Administration-Guide.md` . Pipeline outputs quoted verbatim from lab GitLab runs on 18–19 Oct 2026. Counts measured 19 Oct 2026.